大厂后端工程师社招面试题库（200+题）- 详细来源版

1. Java基础（30题）

1. **JDK和JRE的区别是什么？**
   - **JDK（Java Development Kit）**：Java开发工具包，包含JRE、编译器（javac）、调试器、文档生成工具等，用于开发Java程序。
   - **JRE（Java Runtime Environment）**：Java运行时环境，包含JVM、Java类库和运行所需文件，仅用于运行Java程序。
   - **关系**：JDK包含JRE，JRE包含JVM。
2. **==和equals的区别？** (参考链接)
   - **==**：比较两个对象的引用（地址）是否相同，对于基本数据类型比较值是否相等。
   - **equals()**：比较两个对象的内容是否相等，默认行为与==相同，但String、Integer等类重写了此方法。
   - **注意**：使用equals()前应先判空，避免空指针异常。
3. **final关键字的作用？** (参考链接)
   - **修饰变量**：变量一旦赋值就不能改变，基本类型值不变，引用类型地址不变但内容可变。
   - **修饰方法**：方法不能被子类重写。
   - **修饰类**：类不能被继承，如String、Integer等包装类。
   - **应用场景**：防止意外修改、确保线程安全、提高性能（JIT优化）。
4. **String、StringBuilder、StringBuffer的区别？** (参考链接)
   - **String**：不可变字符串，每次修改都会创建新对象，线程安全但性能较差。
   - **StringBuffer**：可变字符串，线程安全（方法加synchronized），性能中等。
   - **StringBuilder**：可变字符串，非线程安全，性能最好。
   - **选择原则**：单线程字符串拼接用StringBuilder，多线程用StringBuffer，固定字符串用String。
5. **String s = new String("xyz"); 创建了几个对象？**
   - **可能1个或2个对象**：
     - 如果字符串常量池中已存在"xyz"，则只在堆中创建1个新对象（new String()）
     - 如果字符串常量池中不存在"xyz"，则在常量池创建1个，堆中创建1个，共2个对象
   - **常量池机制**："xyz"作为字符串字面量会放入常量池（如果不存在的话）
6. **Object类有哪些方法？** (参考链接)
   - **public final native Class<?> getClass()**：获取对象的运行时类

- **public native int hashCode()**：返回对象的哈希码
- **public boolean equals(Object obj)**：比较对象是否相等
- **protected native Object clone()**：创建并返回对象的副本
- **public String toString()**：返回对象的字符串表示
- **public final native void notify()**：唤醒一个等待线程
- **public final native void notifyAll()**：唤醒所有等待线程
- **public final native void wait(long timeout)**：使当前线程等待指定时间
- **public final void wait(long timeout, int nanos)**：精确时间等待
- **public final native void wait()**：无限期等待
- **protected void finalize()**：对象被垃圾回收前调用

7. **Java中的异常体系？**(参考链接)
   - **Throwable**：异常体系的根类
     - **Error**：系统级错误，如OutOfMemoryError、StackOverflowError，程序无法处理
     - **Exception**：程序可处理的异常
       - **RuntimeException（运行时异常）**：如NullPointerException、ArrayIndexOutOfBoundsException，非检查异常
       - **Checked Exception（检查异常）**：如IOException、SQLException，必须处理或声明抛出
   - **处理方式**：try-catch-finally、throws、throw
8. **throw和throws的区别？**(参考链接)
   - **throw**：关键字，用于在方法体内显式抛出异常实例，如throw new IllegalArgumentException("参数错误")
   - **throws**：关键字，用于方法声明处，声明可能抛出的异常类型，如public void method() throws IOException
   - **作用**：throw是抛出异常对象，throws是声明异常类型
9. **Java中数据类型的转换？**(参考链接)
   - **自动类型转换（隐式）**：从小到大的转换，如int转long、float转double
   - **强制类型转换（显式）**：从大到小的转换，如double转int，需注意精度丢失
   - **转换规则**：byte、short、char → int → long → float → double（容量从小到大）
   - **注意事项**：浮点数转整数会截断小数部分，可能造成数据丢失
10. **Java中的装箱和拆箱？**(参考链接)

- **装箱**：基本数据类型转为包装类，如int转Integer，自动调用valueOf()方法
- **拆箱**：包装类转为基本数据类型，如Integer转int，自动调用intValue()方法
- **缓存机制**：Integer等包装类在-128到127范围内有缓存，超出范围会创建新对象
- **性能影响**：频繁装箱拆箱会产生临时对象，影响性能

11. **重载和重写的区别？** (参考链接)

- **重载（Overload）**：
  - 同一类中，方法名相同，参数列表不同（类型、数量、顺序）
  - 返回类型可不同，访问修饰符无要求
  - 发生在编译时，是静态绑定
- **重写（Override）**：
  - 子类中重写父类方法，方法名、参数列表、返回类型相同
  - 访问权限不能更低，抛出异常不能更宽泛
  - 发生在运行时，是动态绑定

12. **抽象类和接口的区别？** (参考链接)

- **抽象类**：用abstract修饰，可包含抽象方法和具体方法，有构造函数，成员变量可为任意类型
- **接口**：用interface修饰，JDK8前只能有抽象方法，JDK8后可有默认方法和静态方法
- **继承**：类只能继承一个抽象类，但可实现多个接口
- **设计理念**：抽象类是“是什么”，接口是“做什么”

13. **Java中创建线程的方式有哪些？**

- **继承Thread类**：重写run()方法，创建实例并调用start()
- **实现Runnable接口**：实现run()方法，传入Thread构造函数
- **实现Callable接口**：实现call()方法（有返回值），配合FutureTask使用
- **使用线程池**：通过ExecutorService提交任务
- **使用定时器**：Timer和ScheduledExecutorService

14. **sleep()和wait()的区别？** (参考链接)

- **所属类**：sleep()属于Thread类，wait()属于Object类
- **锁释放**：sleep()不释放锁，wait()释放锁
- **唤醒方式**：sleep()时间到自动唤醒，wait()需notify()/notifyAll()唤醒
- **使用场景**：sleep()用于暂停线程，wait()用于线程间协作
- **异常**：sleep()抛InterruptedException，wait()需声明此异常

15. **hashCode()和equals()的关系？** (参考链接)

- **约定**：如果两个对象equals()返回true，则它们的hashCode()必须相等
- **反向不成立**：hashCode()相等的对象不一定equals()返回true（哈希碰撞）
- **重要性**：在HashMap、HashSet等集合中，违反此约定会导致数据存储异常
- **实践**：重写equals()时必须重写hashCode()

16. **Java中Comparable和Comparator的区别？** (参考链接)

- **Comparable**：自然排序接口，实现compareTo()方法，修改源码，适用于类本身确定排序规则
- **Comparator**：外部排序接口，实现compare()方法，无需修改源码，适用于多种排序规则
- **使用**：Collections.sort()、Arrays.sort()可接受Comparator
- **灵活性**：Comparator更灵活，可在运行时决定排序规则

17. **Java反射的作用和优缺点？** (参考链接)

- **作用**：运行时获取类信息、创建对象、调用方法、访问字段
- **优点**：实现框架功能、动态代理、依赖注入、配置化编程
- **缺点**：性能开销大、破坏封装性、类型安全检查失效
- **应用场景**：Spring IoC、MyBatis ORM、Jackson JSON转换

18. **Java泛型的实现原理？** (参考链接)

- **类型擦除**：编译后泛型信息被擦除，只保留原始类型
- **桥方法**：编译器自动生成桥方法以保持多态性
- **边界检查**：编译时进行类型检查，运行时无泛型信息
- **局限性**：不能使用基本类型、无法获取泛型实际类型、数组创建限制

19. **Java序列化和反序列化？** (参考链接)

- **序列化**：将对象转换为字节流，实现Serializable接口，使用ObjectOutputStream
- **反序列化**：将字节流恢复为对象，使用ObjectInputStream
- **注意事项**：静态变量不会被序列化，transient关键字可排除字段
- **版本兼容**：serialVersionUID确保版本兼容性

20. **Java深拷贝和浅拷贝的区别？** (参考链接)

- **浅拷贝**：只复制对象本身，不复制对象内部的引用对象，拷贝后对象共享引用
- **深拷贝**：复制对象本身及内部所有引用对象，完全独立
- **实现方式**：浅拷贝用clone()或构造函数，深拷贝需递归复制所有引用对象
- **序列化实现**：通过序列化/反序列化可实现深拷贝

21. **Java中静态变量和实例变量的区别？**

- **静态变量**：属于类，所有实例共享，内存中只有一份，通过类名访问
- **实例变量**：属于对象，每个实例独立，通过对象访问
- **生命周期**：静态变量随类加载而存在，实例变量随对象创建而存在
- **内存分配**：静态变量在方法区，实例变量在堆中

22. **Java中静态方法和实例方法的区别?**

- **静态方法**：属于类，无需实例化即可调用，不能访问实例变量和方法
- **实例方法**：属于对象，需实例化后才能调用，可访问静态和实例成员
- **内存分配**：静态方法在类加载时分配，实例方法在对象创建时分配
- **调用方式**：静态方法通过类名调用，实例方法通过对象调用

23. **Java中内部类的作用?**

- **访问权限**：内部类可访问外部类的所有成员（包括private）
- **封装性**：将相关类组织在一起，隐藏实现细节
- **多重继承**：内部类可继承其他类，同时外部类也可继承其他类
- **类型**：成员内部类、静态内部类、局部内部类、匿名内部类

24. **Java中this和super的作用?**

- **this**：指向当前对象的引用，用于区分成员变量和局部变量、调用构造方法
- **super**：指向父类对象的引用，用于访问父类成员、调用父类构造方法
- **使用场景**：方法参数与成员变量同名、构造方法链式调用、访问父类方法

25. **Java中访问修饰符的区别?**

- **private**：仅在同一类中可见
- **default（包访问）**：同一包中可见
- **protected**：同一包中可见，或不同包子类可见
- **public**：所有地方可见
- **访问级别**：private < default < protected < public

26. **Java中基本数据类型占用的字节数?**

- **byte**：1字节（-128~127）
- **short**：2字节（-32768~32767）
- **int**：4字节（-2^{31}~$2^{31}-1$）
- **long**：8字节（-2^{63}~$2^{63}-1$）
- **float**：4字节（32位IEEE 754标准）
- **double**：8字节（64位IEEE 754标准）
- **char**：2字节（Unicode字符）
- **boolean**：JVM规范未定义大小（通常1位或1字节）

27. **Java中switch语句支持的数据类型?**

- **JDK7以前**：byte、short、char、int及其包装类
- **JDK7**：新增String类型（通过hashCode()和equals()实现）
- **JDK8**：支持Enum类型
- **不支持**：long、float、double及其包装类
- **实现原理**：int类型使用tableswitch，稀疏数据使用lookupswitch

28. **Java中foreach循环的实现原理?**

- **本质**：foreach是语法糖，编译后转换为Iterator迭代器遍历
- **实现条件**：对象需实现Iterable接口或数组类型
- **优势**：代码简洁、类型安全、防止越界
- **注意事项**：遍历时不可修改集合大小，否则抛ConcurrentModificationException

29. **Java中try-catch-finally的执行顺序?**

- **正常情况**：try → finally → (正常结束或return)
- **异常情况**：try → catch → finally → (异常处理结束)
- **return处理**：
  - try/catch中有return，先执行return表达式，再执行finally，最后return
  - finally中return会覆盖try/catch中的return
- **最佳实践**：finally中避免return、throw语句

30. **Java中try-with-resources的原理?**

- **语法**：try(Resource res = new Resource()) { }，Resource需实现AutoCloseable接口
- **原理**：编译器自动生成finally块，确保资源自动关闭
- **优势**：简化资源管理、防止资源泄露、即使异常也能关闭资源
- **底层**：通过invokevirtual调用close()方法，处理关闭异常

2. JVM（20题）

31. **JVM内存区域划分?** (参考链接)

- **程序计数器**：线程私有，记录当前线程执行的字节码行号，唯一不发生OOM的区域
- **Java虚拟机栈**：线程私有，存储局部变量表、操作数栈、动态链接、方法出口等信息
- **本地方法栈**：线程私有，为Native方法服务
- **Java堆**：线程共享，存放对象实例，是GC主要区域，分为新生代和老年代
- **方法区**：线程共享，存储类信息、常量、静态变量、即时编译器编译后的代码等
- **运行时常量池**：方法区的一部分，存放编译期生成的字面量和符号引用

- **建议配图**：JVM内存结构图，清晰展示各个内存区域的位置和关系

32. **方法区和永久代的区别?**

- **永久代（PermGen）**：JDK8之前的实现，使用固定的内存空间，容易发生OOM
- **元空间（Metaspace）**：JDK8之后的实现，使用本地内存，理论上只受物理内存限制
- **改进**：元空间减少了OOM风险，提高了类卸载效率，更适合动态语言
- **参数调整**：-XX:MaxPermSize（JDK8前）→ -XX:MaxMetaspaceSize（JDK8后）

33. **堆和栈的区别?** (参考链接)

- **存储内容**：堆存储对象实例，栈存储局部变量、方法调用信息
- **生命周期**：堆中对象生命周期由GC管理，栈中变量随方法调用结束
- **线程共享**：堆是线程共享，栈是线程私有
- **内存分配**：堆分配相对复杂，有GC机制；栈分配简单，速度快
- **内存大小**：堆通常较大，栈较小

34. **垃圾回收算法有哪些?** (参考链接)

- **标记-清除算法**：标记存活对象，清除未标记对象，会产生内存碎片
- **复制算法**：将内存分为两块，只使用一块，GC时将存活对象复制到另一块
- **标记-整理算法**：标记存活对象，将存活对象向一端移动，清理边界外内存
- **分代收集算法**：根据对象存活周期将内存分代，不同代使用不同算法
- **应用场景**：新生代使用复制算法，老年代使用标记-清除或标记-整理

35. **垃圾收集器有哪些？CMS和G1的区别?** (参考链接)

- **Serial**：单线程，新生代，简单高效
- **ParNew**：多线程版本Serial，新生代
- **Parallel Scavenge**：关注吞吐量，新生代
- **Serial Old**：单线程，老年代
- **Parallel Old**：多线程，老年代
- **CMS**：关注停顿时间，老年代，初始标记、并发标记、重新标记、并发清除
- **G1**：面向服务端，整体基于标记-整理，局部基于复制，可预测停顿

36. **如何判断对象是否存活?** (参考链接)

- **引用计数法**：给对象添加引用计数器，有引用+1，引用失效-1，计数为0则死亡，无法解决循环引用
- **可达性分析算法**：以GC Roots为起点，向下搜索，不可达对象即为垃圾

- **GC Roots包括**：虚拟机栈中引用的对象、方法区中类静态属性引用的对象、方法区中常量引用的对象、本地方法栈中JNI引用的对象
- **四种引用**：强引用、软引用、弱引用、虚引用

37. **类加载过程？** (参考链接)

- **加载**：通过类的全限定名获取类的二进制字节流，将其转换为方法区的运行时数据结构
- **验证**：确保Class文件字节流符合当前虚拟机要求，包括文件格式、元数据、字节码、符号引用验证
- **准备**：为类变量分配内存并设置初始值（零值），不执行初始化代码
- **解析**：将符号引用替换为直接引用
- **初始化**：执行类构造器()方法，执行静态变量赋值和静态代码块

38. **类加载器的种类？** (参考链接)

- **Bootstrap ClassLoader**：启动类加载器，加载<JAVA_HOME>/lib目录下的核心类库
- **Extension ClassLoader**：扩展类加载器，加载<JAVA_HOME>/lib/ext目录下的类库
- **Application ClassLoader**：应用程序类加载器，加载用户类路径上的类库
- **Custom ClassLoader**：自定义类加载器，用户可扩展
- **关系**：双亲委派模型，优先委托父类加载器加载

39. **双亲委派模型？** (参考链接)

- **定义**：当一个类加载器收到类加载请求时，先委托给父类加载器去完成，顶层为Bootstrap ClassLoader
- **流程**：先检查类是否已被加载，若未加载则委托父类，若父类无法完成则自己尝试加载
- **优势**：保证类的唯一性，避免重复加载，保护核心API不被篡改
- **破坏**：JNDI、OSGi、热部署等场景需要打破双亲委派

40. **如何排查内存泄漏？** (参考链接)

- **现象识别**：频繁Full GC、GC后内存仍然很高、应用响应缓慢
- **工具使用**：jstat观察GC情况、jmap生成堆dump、MAT/Eclipse MAT分析堆内存
- **常见原因**：静态集合类持有对象引用、监听器未注销、内部类持有外部类引用、数据库连接未关闭
- **解决方法**：及时释放资源、使用弱引用、避免创建不必要的对象、定期清理缓存

41. **常用的JVM参数有哪些？** (参考链接)

- **堆内存**：-Xms设置初始堆大小，-Xmx设置最大堆大小，-Xmn设置新生代大小
- **栈内存**：-Xss设置栈大小
- **GC参数**：-XX:+UseG1GC选择G1收集器，-XX:MaxGCPauseMillis设置最大GC停顿时间

- **调试参数**：-XX:+PrintGCDetails打印GC详细信息，-XX:+HeapDumpOnOutOfMemoryError发生OOM时生成dump
- **性能参数**：-XX:NewRatio设置新生代与老年代比例，-XX:SurvivorRatio设置Eden与Survivor比例

42. **如何分析堆dump文件？**

- **生成方式**：-XX:+HeapDumpOnOutOfMemoryError、jmap -dump:format=b,file=heap.hprof
- **分析工具**：MAT（Memory Analyzer Tool）、JProfiler、VisualVM
- **分析步骤**：打开dump文件、查看直方图、分析支配树、查找泄漏路径
- **关键指标**：对象数量、占用内存、GC Root路径

43. **Full GC频繁的原因和解决方案？** (参考链接)

- **原因分析**：
  - 老年代空间不足：对象提前进入老年代、大对象直接进入老年代
  - 元空间不足：大量动态代理、反射生成类
  - 内存泄漏：集合类持有对象引用、缓存未清理
- **解决方案**：
  - 调整堆内存大小、优化对象生命周期、及时释放资源、选择合适GC算法

44. **如何进行JVM调优？** (参考链接)

- **明确目标**：降低GC频率、减少GC停顿时间、提高吞吐量
- **监控指标**：GC频率、GC时间、堆内存使用率、CPU使用率
- **调优步骤**：
  - 监控现状、分析瓶颈、调整参数、验证效果、持续优化
- **常用参数**：选择合适的GC收集器、调整堆内存比例、设置合理的新生代大小

45. **什么情况下会发生栈溢出？**

- **递归调用过深**：没有终止条件的递归或递归层级过深
- **方法调用链过长**：相互调用形成死循环
- **栈内存不足**：-Xss参数设置过小
- **解决方法**：增加栈内存大小、优化算法减少递归深度、使用循环代替递归
- **异常类型**：StackOverflowError（单线程）或OutOfMemoryError（多线程）

46. **什么是逃逸分析？** (参考链接)

- **定义**：分析对象的动态作用域，判断对象是否会被外部方法或线程访问
- **逃逸类型**：全局逃逸（被其他线程访问）、方法逃逸（被其他方法访问）、没有逃逸
- **优化技术**：

    - 栈上分配：没有逃逸的对象分配在栈上，随方法结束自动回收
    - 同步消除：没有逃逸的对象不需要同步
    - 标量替换：将对象分解为标量变量，减少堆内存分配

47. **什么是TLAB?** (参考链接) [(简单易懂示例)](file:///Users/longyun/work/interview-prep/src/main/java/com/example/interview/demo/SimpleTLABDemo.java)

- **全称**：Thread Local Allocation Buffer，线程本地分配缓冲区
- **通俗理解**：给每个线程分配一个私人的小仓库，创建对象时不用排队
- **目的**：减少多线程分配内存时的同步开销
- **工作原理**：每个线程有自己的内存缓冲区，直接从自己区域分配对象
- **优势**：提高对象分配效率，减少锁竞争，提升多线程性能
- **相关参数**：-XX:+UseTLAB启用TLAB（默认已开启）
- **Java示例**： `demo/SimpleTLABDemo.java` （推荐入门）

48. **Java对象的创建过程?** (参考链接) [(Java示例代码)](file:///Users/longyun/work/interview-prep/src/main/java/com/example/interview/demo/SimpleObjectCreationDemo.java)

- **类加载检查**：检查类是否已加载，如未加载则执行类加载过程
- **内存分配**：在Eden区分配内存，使用指针碰撞或空闲列表方式（TLAB优化）
- **内存初始化**：将分配的内存空间初始化为零值（不包括对象头）
- **设置对象头**：设置对象的元数据信息，如哈希码、GC分代年龄、锁标志等
- **执行init方法**：执行方法，按顺序：父类构造器→实例代码块→构造方法
- **对象创建完成**：返回对象地址
- **Java示例**： `demo/ObjectCreationDemo.java` （详细版） `demo/SimpleObjectCreationDemo.java` （面试版）

49. **对象的内存布局?** (参考链接)

- **对象头（Header）**：
    - Mark Word：存储哈希码、GC分代年龄、锁标志、线程持有的锁等信息
    - Class Pointer：指向类元数据的指针
    - Array Length（数组对象）：数组长度
- **实例数据（Instance Data）**：对象真正存储的有效信息
- **对齐填充（Padding）**：保证对象大小为8字节的倍数

50. **如何分析GC日志?**

- **日志格式**：[GC类型][开始时间][堆内存变化][GC时间]
- **关键信息**：GC触发原因、GC前后内存使用情况、GC耗时

- **分析要点**：
  - GC频率：频繁GC可能是内存不足或内存泄漏
  - GC时间：长时间停顿影响用户体验
  - 内存回收效果：回收后内存使用率是否合理
- **工具辅助**：GCeasy.io、GCViewer等可视化分析工具

3. 多线程与并发（30题）

51. **进程和线程的区别？** (参考链接)

- **进程**：操作系统资源分配的基本单位，拥有独立的内存空间，进程间相互隔离
- **线程**：CPU调度的基本单位，共享进程的内存空间，线程间通信更容易
- **资源开销**：进程创建销毁开销大，线程开销小
- **通信方式**：进程间通信需要IPC机制，线程间可直接共享内存
- **并发性**：进程间并发，线程间并行或并发

52. **创建线程的几种方式？** (参考链接) [(Java示例代码)](file:///Users/longyun/work/interview-prep/src/main/java/com/example/interview/demo/SimpleThreadCreationDemo.java)

- **继承Thread类**：重写run()方法，创建实例并调用start()（简单但有继承限制）
- **实现Runnable接口**：实现run()方法，传入Thread构造函数（推荐方式）
- **实现Callable接口**：实现call()方法（有返回值），配合FutureTask使用
- **使用线程池**：通过Executors工厂类创建线程池（性能好，推荐）
- **使用CompletableFuture**：JDK8新增，支持异步编程（现代推荐）
- **Java示例**： `demo/ThreadCreationDemo.java` （详细版） `demo/SimpleThreadCreationDemo.java` （面试版）

53. **线程的生命周期？** (参考链接)

- **NEW**：新建状态，线程对象已创建但未启动
- **RUNNABLE**：就绪状态，线程在JVM中执行
- **BLOCKED**：阻塞状态，等待获取monitor锁
- **WAITING**：等待状态，等待其他线程通知
- **TIMED_WAITING**：超时等待状态，等待指定时间
- **TERMINATED**：终止状态，线程执行完毕

💡 **配套代码演示**：ThreadLifecycleDemo.java - 完整演示线程6种状态的转换过程

54. **线程池的核心参数？** (参考链接)

- **corePoolSize**：核心线程数，即使空闲也不会被回收
- **maximumPoolSize**：最大线程数，线程池允许的最大线程数量
- **keepAliveTime**：非核心线程空闲存活时间
- **unit**：存活时间单位
- **workQueue**：阻塞队列，存储等待执行的任务
- **threadFactory**：线程工厂，创建新线程
- **handler**：拒绝策略，无法执行任务时的处理方式
- **建议配图**：线程池工作原理图，展示任务提交、排队和执行的完整流程

55. **线程池的拒绝策略？** (参考链接)

- **AbortPolicy**：默认策略，直接抛出RejectedExecutionException异常
- **CallerRunsPolicy**：由调用线程处理该任务，提供调节机制
- **DiscardPolicy**：丢弃任务，不抛异常
- **DiscardOldestPolicy**：丢弃队列头部任务，然后重新提交当前任务

💡 **配套代码演示**：ThreadPoolRejectionStrategiesDemo.java - 完整演示4种拒绝策略的实际效果

56. **线程池的种类？** (参考链接)

- **FixedThreadPool**：固定大小线程池，core=max，无界队列
- **CachedThreadPool**：缓存线程池，core=0，max=Integer.MAX_VALUE
- **ScheduledThreadPool**：定时任务线程池，支持定时和周期性任务
- **SingleThreadExecutor**：单线程线程池，确保任务按顺序执行

57. **synchronized的实现原理？** (参考链接)

- **JVM层面**：通过monitorenter和monitorexit指令实现
- **对象头**：包含Mark Word，存储锁信息
- **锁升级**：无锁→偏向锁→轻量级锁→重量级锁
- **Monitor**：每个对象都有一个Monitor，线程通过竞争Monitor来获得锁
- **可重入性**：通过计数器实现，每次获得锁+1，释放-1

58. **volatile关键字的作用？** (参考链接)

- **可见性**：修改volatile变量后立即刷新到主内存
- **禁止指令重排序**：保证代码执行顺序
- **不保证原子性**：仅保证单次读/写的原子性，复合操作仍需同步
- **内存屏障**：通过内存屏障实现可见性和有序性
- **使用场景**：状态标志、双重检查锁定、单例模式

59. **CAS的原理？** (参考链接)

- **全称**：Compare And Swap，比较并交换
- **操作**：CAS(V, A, B)，V为内存位置，A为预期值，B为新值
- **过程**：比较V和A，相等则更新V为B，否则重试
- **优点**：无锁，避免线程阻塞和唤醒的开销
- **缺点**：ABA问题、循环时间长、只能保证一个共享变量的原子操作

60. **AQS的原理？** (参考链接)

- **全称**：AbstractQueuedSynchronizer，队列同步器
- **核心思想**：使用volatile int state表示同步状态，FIFO队列管理等待线程
- **实现方式**：继承AQS，重写tryAcquire/tryRelease等方法
- **应用场景**：ReentrantLock、CountDownLatch、Semaphore等
- **状态管理**：通过CAS操作state，保证状态变更的原子性

61. **ReentrantLock和synchronized的区别？** (参考链接)

- **实现层面**：synchronized是关键字，ReentrantLock是API
- **公平性**：synchronized非公平，ReentrantLock可选择公平/非公平
- **响应中断**：synchronized不可中断，ReentrantLock可响应中断
- **超时获取**：synchronized不支持，ReentrantLock支持超时获取
- **条件队列**：synchronized单一，ReentrantLock可创建多个Condition

62. **CountDownLatch和CyclicBarrier的区别？** (参考链接)

- **CountDownLatch**：一个或多个线程等待其他线程完成操作
- **CyclicBarrier**：一组线程互相等待到达某个屏障点
- **重用性**：CountDownLatch不可重用，CyclicBarrier可重用
- **计数**：CountDownLatch减计数，CyclicBarrier加计数
- **用途**：CountDownLatch用于等待，CyclicBarrier用于同步

63. **Semaphore的作用？** (参考链接)

- **信号量**：控制同时访问资源的线程数量
- **许可机制**：通过acquire()获取许可，release()释放许可
- **应用场景**：限制并发线程数、资源池管理
- **公平性**：可选择公平或非公平模式
- **使用方式**：初始化指定许可数量，线程获取许可后执行，完成后释放

64. **ThreadLocal的实现原理？** (参考链接)

- **作用**：为每个线程提供独立的变量副本，实现线程隔离
- **实现**：Thread类中包含ThreadLocalMap，键为ThreadLocal实例，值为变量副本
- **存储**：每个线程维护自己的变量副本，互不影响
- **使用**：set()设置值，get()获取值，remove()移除值
- **内存结构**：ThreadLocalMap使用弱引用存储ThreadLocal作为key

65. **ThreadLocal内存泄漏问题？** (参考链接)

- **原因**：ThreadLocalMap中key使用弱引用，value使用强引用
- **问题**：ThreadLocal对象被回收后，key为null但value未被回收
- **影响**：导致Entry无法被访问但无法被回收，造成内存泄漏
- **解决方案**：使用完ThreadLocal后手动调用remove()方法
- **预防措施**：在try-finally中使用ThreadLocal

66. **什么是原子类？** (参考链接)

- **定义**：基于CAS实现的一组原子操作类，保证操作的原子性
- **常用类**：AtomicInteger、AtomicLong、AtomicBoolean、AtomicReference等
- **实现原理**：底层使用Unsafe类提供的CAS操作
- **优势**：比synchronized性能更好，避免线程阻塞
- **应用场景**：计数器、状态标志、无锁数据结构

67. **Java中的锁有哪些？** (参考链接)

- **synchronized**：内置锁，JVM实现
- **ReentrantLock**：可重入锁，API实现，功能更丰富
- **ReadWriteLock**：读写锁，读读不互斥，读写/写写互斥
- **StampedLock**：JDK8新增，性能更好的读写锁
- **公平锁/非公平锁**：按申请顺序获取/随机获取
- **可重入锁/非可重入锁**：可多次获取/不可重复获取

68. **乐观锁和悲观锁的区别？** (参考链接)

- **悲观锁**：假设会发生冲突，访问数据前加锁，如synchronized、ReentrantLock
- **乐观锁**：假设不会发生冲突，访问时不加锁，更新时检查是否被修改
- **实现方式**：悲观锁通过阻塞实现，乐观锁通过CAS实现
- **适用场景**：悲观锁适用于写多读少，乐观锁适用于读多写少
- **性能特点**：悲观锁有线程切换开销，乐观锁避免线程阻塞

69. **什么是可重入锁？** (参考链接)

- **定义**：同一个线程可以多次获取同一把锁
- **实现**：通过计数器实现，获取锁+1，释放锁-1，为0时完全释放
- **优势**：避免死锁，允许同一线程的递归调用
- **示例**：synchronized和ReentrantLock都是可重入锁
- **重要性**：防止线程因重复获取锁而阻塞

70. **什么是公平锁和非公平锁？** (参考链接)

- **公平锁**：按申请顺序获取锁，先申请的线程先获得锁
- **非公平锁**：不按顺序，允许插队，可能造成线程饥饿
- **性能差异**：非公平锁性能更好，减少线程阻塞和唤醒次数
- **实现方式**：ReentrantLock(true)公平锁，ReentrantLock(false)非公平锁
- **默认选择**：synchronized和ReentrantLock默认非公平

71. **什么是读写锁？** (参考链接)

- **定义**：将读操作和写操作分开，允许多个线程同时读取
- **规则**：读读不互斥、读写互斥、写写互斥
- **优势**：提高读多写少场景的并发性能
- **实现**：ReadWriteLock接口，ReentrantReadWriteLock实现类
- **应用场景**：缓存系统、配置信息读取

72. **synchronized锁升级过程？** (参考链接)

- **无锁状态**：对象刚创建时的状态
- **偏向锁**：只有一个线程访问时，JVM将对象头偏向该线程
- **轻量级锁**：多线程交替执行同步块，无竞争时
- **重量级锁**：多线程竞争激烈时，线程阻塞
- **升级目的**：减少锁的性能消耗，提高执行效率

73. **Java中wait()和sleep()的区别？** (参考链接)

- **所属类**：wait()属于Object类，sleep()属于Thread类
- **锁释放**：wait()释放锁，sleep()不释放锁
- **唤醒方式**：wait()需notify()/notifyAll()唤醒，sleep()时间到自动唤醒
- **使用条件**：wait()必须在同步代码块中，sleep()任意位置
- **异常处理**：wait()需声明InterruptedException，sleep()需捕获

74. **notify()和notifyAll()的区别？** (参考链接)

- **notify()**：唤醒一个等待线程，具体唤醒哪个不确定
- **notifyAll()**：唤醒所有等待线程，由JVM选择一个获得锁
- **选择依据**：notify()用于单一条件变化，notifyAll()用于多个条件变化
- **性能影响**：notify()性能更好，但可能导致遗漏；notifyAll()更安全
- **使用场景**：根据业务逻辑选择合适的唤醒方式

75. **什么是死锁？如何避免？** (参考链接)

- **定义**：多个线程因争夺资源而造成的一种互相等待的僵局
- **产生条件**：互斥条件、请求和保持、不剥夺条件、循环等待
- **避免方法**：
  - 破坏请求和保持：一次性申请所有资源
  - 破坏循环等待：按固定顺序获取锁
  - 使用超时机制：tryLock(timeout)
  - 使用JDK工具检测死锁

76. **线程安全的单例模式写法？** (参考链接)

- **双重检查锁定**：

```java
public class Singleton {
    private static volatile Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

- **静态内部类**：利用类加载机制保证线程安全

77. **Java内存模型(JMM)？** (参考链接)

- **定义**：屏蔽各种硬件和操作系统的内存访问差异

- **目标**：确保程序在各种平台上对内存的访问都能得到一致的效果
- **核心概念**：主内存、工作内存、内存间交互操作
- **三大特性**：原子性、可见性、有序性
- **实现机制**：通过happens-before原则、内存屏障等实现

78. **happens-before原则？** (参考链接)

- **定义**：两个操作间的偏序关系，如果A happens-before B，则A的执行结果对B可见
- **八大原则**：
  - 程序次序规则：同一线程中，代码顺序决定happens-before
  - 锁定规则：解锁(happen)发生在随后的加锁之前
  - volatile变量规则：对volatile变量的写操作happens-before读操作
  - 线程启动规则：线程start()方法happens-before线程内的动作
  - 线程终结规则：线程中所有操作happens-before线程的终止检测
  - 中断规则：发送中断happens-before被中断线程的代码
  - 对象终结规则：对象初始化完成happens-beforefinalize()方法
  - 传递性：A happens-before B，B happens-before C，则A happens-before C

79. **ConcurrentHashMap的实现原理？** (参考链接)

- **JDK7**：Segment分段锁，将Hash表分成多个Segment
- **JDK8**：CAS+synchronized，Node数组+链表+红黑树
- **核心思想**：使用volatile保证可见性，synchronized保证原子性
- **优化点**：
  - 使用synchronized关键字锁住链表头节点，减少锁粒度
  - 引入红黑树优化长链表性能
  - 使用sizeCtl控制扩容

80. **BlockingQueue的实现类？** (参考链接)

- **ArrayBlockingQueue**：有界阻塞队列，数组实现，FIFO，容量固定
- **LinkedBlockingQueue**：可选有界阻塞队列，链表实现，FIFO
- **PriorityBlockingQueue**：无界阻塞队列，优先级队列实现
- **DelayQueue**：无界阻塞队列，按延迟时间排序
- **SynchronousQueue**：同步移交队列，不存储元素
- **LinkedTransferQueue**：链表实现的无界阻塞队列，支持transfer操作

4. Java集合（20题）

81. **ArrayList和LinkedList的区别？** (参考链接)

- **底层实现**：ArrayList基于动态数组，LinkedList基于双向链表
- **查询效率**：ArrayList O(1)，LinkedList O(n)
- **增删效率**：ArrayList O(n)（涉及元素移动），LinkedList O(1)（找到位置后）
- **内存占用**：ArrayList连续内存，LinkedList额外存储前后节点引用
- **适用场景**：ArrayList适合查询多、增删少；LinkedList适合增删多、查询少

82. **HashMap的实现原理？** (参考链接)

- **JDK1.7**：数组+链表，Entry数组存储，链表解决哈希冲突
- **JDK1.8**：数组+链表+红黑树，Node数组存储，链表长度>8转红黑树
- **存储结构**：通过哈希值确定数组索引，相同哈希值的元素形成链表
- **扩容机制**：容量达到阈值时，数组大小翻倍，元素重新散列
- **线程安全**：非线程安全，多线程环境下使用ConcurrentHashMap

83. **HashMap在JDK1.8中的改进？** (参考链接)

- **红黑树优化**：链表长度>8且数组长度>64时转为红黑树，查找效率从O(n)提升到O(logn)
- **扩容优化**：多线程扩容时采用尾插法，避免死循环问题
- **哈希算法优化**：减少哈希冲突，提高分布均匀性
- **存储结构**：Node数组+链表+红黑树
- **性能提升**：在高冲突场景下性能显著提升

84. **为什么HashMap的数组容量是2的幂次方？**

- **索引计算**：通过 (n-1) & hash 计算索引，当n为2的幂时，n-1的二进制全为1
- **均匀分布**：&运算等效于取模运算，但性能更好，能更好地分散元素
- **减少冲突**：均匀分布减少哈希冲突，提高查找效率
- **位运算优化**：2的幂次方便于位运算，提高计算效率

85. **HashMap的扩容机制？** (参考链接)

- **触发条件**：当元素数量超过阈值（capacity * loadFactor）时触发扩容
- **扩容大小**：容量扩大为原来的2倍
- **JDK1.7**：重新计算哈希值，将元素放入新数组
- **JDK1.8**：利用resize掩码，通过位运算判断元素位置不变或移动到原位置+旧容量
- **性能影响**：扩容是耗时操作，应合理设置初始容量

86. **什么是哈希冲突？如何解决？** (参考链接)

- **定义**：不同的键通过哈希函数得到相同的哈希值，映射到同一位置
- **解决方法**：
  - 链地址法：相同位置的元素用链表连接（HashMap采用）
  - 开放寻址法：探测下一个空位置存储
  - 再哈希法：使用另一个哈希函数计算位置
- **影响**：降低查找效率，理想情况O(1)变为O(n)

87. **ConcurrentHashMap的实现原理？** (参考链接)

- **JDK1.7**：分段锁机制，Segment数组，每个Segment相当于一个小HashMap
- **JDK1.8**：CAS+synchronized，Node数组+链表+红黑树，synchronized锁住链表头节点
- **线程安全**：通过CAS操作和synchronized保证并发安全
- **性能优化**：减少锁粒度，提高并发性能
- **扩容机制**：多线程协助扩容，提高扩容效率

88. **HashSet的实现原理？** (参考链接)

- **底层实现**：基于HashMap，HashSet内部维护一个HashMap实例
- **存储方式**：元素作为HashMap的key，固定值作为value
- **唯一性**：利用HashMap的key唯一性保证元素唯一
- **操作方法**：add()对应HashMap的put()，remove()对应remove()
- **性能特点**：查询、插入、删除时间复杂度O(1)

89. **LinkedHashMap的实现原理？** (参考链接)

- **继承关系**：继承HashMap，扩展了双向链表功能
- **存储结构**：HashMap的数组+链表+红黑树，额外维护双向链表
- **访问顺序**：可按插入顺序或访问顺序排序
- **实现方式**：重写HashMap的newNode()等方法，维护链表关系
- **应用场景**：LRU缓存实现

90. **TreeMap的实现原理？** (参考链接)

- **底层结构**：基于红黑树实现的有序Map
- **排序特性**：根据key的自然顺序或自定义比较器排序
- **时间复杂度**：查找、插入、删除均为O(logn)
- **实现方式**：通过红黑树的旋转和着色维持平衡
- **应用场景**：需要排序的键值对存储

91. **Collection和Collections的区别？** (参考链接)

- **Collection**：集合顶层接口，定义集合的基本操作，如List、Set、Queue
- **Collections**：工具类，提供静态方法操作Collection，如排序、同步包装、查找等
- **性质差异**：Collection是接口，Collections是类
- **功能不同**：Collection定义集合规范，Collections提供集合操作工具

92. **Iterator和ListIterator的区别？** (参考链接)

- **适用范围**：Iterator适用于所有Collection，ListIterator仅适用于List
- **遍历方向**：Iterator只能向前遍历，ListIterator可双向遍历
- **修改能力**：ListIterator除了remove()还能add()和set()
- **获取方式**：Iterator通过iterator()获取，ListIterator通过listIterator()获取
- **位置信息**：ListIterator可获取当前位置索引

93. **fail-fast和fail-safe的区别？** (参考链接)

- **fail-fast**：快速失败机制，遍历时修改集合抛ConcurrentModificationException
- **fail-safe**：安全失败机制，基于集合副本遍历，修改不影响遍历
- **实现方式**：fail-fast通过modCount检查，fail-safe通过复制集合
- **性能差异**：fail-fast性能好但不安全，fail-safe安全但消耗内存
- **典型例子**：ArrayList等使用fail-fast，CopyOnWriteArrayList使用fail-safe

94. **ArrayList扩容机制？** (参考链接)

- **触发条件**：添加元素时容量不足
- **扩容大小**：新容量为原容量的1.5倍（右移一位加原值）
- **实现过程**：创建新数组，复制原数组元素，替换引用
- **性能影响**：扩容涉及数组复制，是耗时操作
- **优化建议**：合理设置初始容量避免频繁扩容

95. **Vector和ArrayList的区别？** (参考链接)

- **线程安全**：Vector线程安全（方法加synchronized），ArrayList非线程安全
- **扩容机制**：Vector扩容2倍，ArrayList扩容1.5倍
- **性能差异**：Vector因同步开销性能较低
- **历史差异**：Vector较早，ArrayList是Collections框架一部分
- **推荐使用**：多线程使用Collections.synchronizedList或CopyOnWriteArrayList

96. **HashMap和HashTable的区别？** (参考链接)

- **线程安全**：HashTable线程安全，HashMap非线程安全
- **效率**：HashMap效率更高，HashTable因同步开销大
- **Null值**：HashMap允许key和value为null，HashTable不允许
- **继承**：HashMap继承AbstractMap，HashTable继承Dictionary
- **推荐使用**：多线程使用ConcurrentHashMap

97. **什么是红黑树？** (参考链接)

- **定义**：自平衡二叉查找树，每个节点包含颜色属性（红或黑）
- **五大性质**：
  - 节点是红色或黑色
  - 根节点是黑色
  - 所有叶子节点（NIL）是黑色
  - 红节点的子节点必须是黑色
  - 从任一节点到其每个叶子的所有路径都包含相同数目的黑色节点
- **操作复杂度**：查找、插入、删除均为O(logn)
- **应用场景**：TreeMap、TreeSet、HashMap（链表转红黑树）

98. **如何实现一个简单的HashMap？**

- **数据结构**：数组+链表，Node类包含key、value、hash、next字段
- **核心方法**：
  - put(K key, V value)：计算hash，找到索引，处理冲突，插入或更新
  - get(Object key)：计算hash，找到索引，遍历链表查找
  - resize()：扩容时创建新数组，重新散列所有元素
- **哈希函数**：使用扰动函数减少冲突
- **扩容阈值**：达到负载因子时触发扩容

99. **ArrayList线程安全吗？如何实现线程安全？** (参考链接)

- **线程安全**：ArrayList不是线程安全的，并发修改可能引发问题
- **线程安全方法**：
  - Collections.synchronizedList(new ArrayList<>())：返回同步包装类
  - CopyOnWriteArrayList：写时复制，读操作无锁
  - 使用并发容器：如Vector（不推荐）
- **推荐方案**：CopyOnWriteArrayList适合读多写少场景

100. **如何遍历一个Collection而不出现并发修改异常？**

- **迭代器同步**：使用Collections.synchronizedCollection()包装

- **复制遍历**：遍历前复制集合副本
- **快照遍历**：使用CopyOnWriteArrayList等快照容器
- **锁机制**：遍历时加锁保护
- **Stream API**：使用并行流处理，但需注意数据一致性

5. 数据库（25题）

101. **MySQL的存储引擎有哪些？** (参考链接)

- **InnoDB**：支持事务、行级锁、外键，支持MVCC，适合高并发场景
- **MyISAM**：不支持事务，表级锁，全文索引支持好，适合读多写少场景
- **Memory**：数据存储在内存中，速度快，重启后数据丢失
- **Archive**：压缩存储，适合存储日志和历史数据
- **Blackhole**：写入的数据会消失，类似/dev/null，用于复制过滤
- **CSV**：以CSV格式存储数据

102. **InnoDB和MyISAM的区别？** (参考链接)

- **事务支持**：InnoDB支持事务，MyISAM不支持
- **锁机制**：InnoDB支持行级锁，MyISAM只支持表级锁
- **外键支持**：InnoDB支持外键，MyISAM不支持
- **崩溃恢复**：InnoDB支持崩溃恢复，MyISAM不支持
- **全文索引**：MyISAM支持全文索引，InnoDB从5.6开始支持
- **存储结构**：InnoDB存储表数据和索引，MyISAM分别存储

103. **什么是事务？事务的特性？** (参考链接)

- **定义**：事务是逻辑上的一组操作，要么全部执行，要么全部不执行
- **ACID特性**：
  - **原子性（Atomicity）**：事务是最小的执行单位，不可分割
  - **一致性（Consistency）**：事务执行前后数据保持一致状态
  - **隔离性（Isolation）**：并发事务之间相互隔离，互不干扰
  - **持久性（Durability）**：事务提交后数据永久保存

104. **数据库的隔离级别？** (参考链接)

- **READ UNCOMMITTED（读未提交）**：最低隔离级别，可能产生脏读、不可重复读、幻读
- **READ COMMITTED（读已提交）**：解决脏读，但可能产生不可重复读、幻读
- **REPEATABLE READ（可重复读）**：解决脏读、不可重复读，可能产生幻读（MySQL解决了幻读）

- **SERIALIZABLE（串行化）**：最高隔离级别，解决所有问题，但性能最差

105. **什么是脏读、幻读、不可重复读？** (参考链接)

- **脏读**：一个事务读取了另一个未提交事务的数据
- **不可重复读**：一个事务内多次读取同一数据，结果不同（其他事务修改了数据）
- **幻读**：一个事务内按相同条件查询，结果集数量不同（其他事务插入了数据）
- **区别**：不可重复读针对UPDATE，幻读针对INSERT/DELETE

106. **MySQL的索引类型？** (参考链接)

- **按功能分类**：普通索引、唯一索引、主键索引、全文索引
- **按结构分类**：B+树索引、Hash索引、R树索引
- **按组合分类**：单列索引、联合索引（复合索引）
- **按物理分类**：聚簇索引、非聚簇索引
- **按存储分类**：聚集索引、辅助索引

107. **B+树索引的原理？** (参考链接)

- **结构特点**：所有数据存储在叶子节点，叶子节点间有指针连接
- **优势**：
  - 支持范围查询
  - 查询稳定，所有查询都需要到叶子节点
  - 减少磁盘I/O，每次读取一个节点
- **分裂合并**：节点满时分裂，节点半空时合并
- **高度平衡**：保持树的高度平衡，保证查询效率

108. **什么是聚簇索引和非聚簇索引？** (参考链接)

- **聚簇索引**：
  - 数据行和索引存储在一起
  - 一个表只能有一个聚簇索引
  - 主键索引通常是聚簇索引
  - 叶子节点直接存储数据行
- **非聚簇索引**：
  - 索引和数据行分开存储
  - 一个表可以有多个非聚簇索引
  - 叶子节点存储主键值
  - 需要回表查询

109. **什么情况下索引会失效？** (参考链接)

- **LIKE查询**：LIKE '%value%'，前缀模糊查询
- **函数操作**：对索引列进行函数操作
- **类型转换**：索引列存在隐式类型转换
- **OR操作**：OR连接的条件部分无索引
- **NOT操作**：NOT IN、!=、<>等否定操作
- **联合索引**：不满足最左前缀原则
- **数据分布**：查询条件返回数据过多

110. **如何优化SQL查询？** (参考链接)

- **索引优化**：
  - 为WHERE、ORDER BY、GROUP BY字段建立索引
  - 合理使用联合索引
  - 避免索引失效
- **查询优化**：
  - 避免SELECT *
  - 使用LIMIT分页
  - 优化JOIN查询
  - 避免在WHERE子句中使用函数

111. **什么是慢查询？** (参考链接)

- **定义**：执行时间超过指定阈值的SQL语句
- **默认阈值**：long_query_time参数，默认10秒
- **开启方式**：设置slow_query_log=ON
- **记录位置**：记录到慢查询日志文件
- **分析工具**：mysqldumpslow、pt-query-digest
- **优化意义**：定位性能瓶颈，提升数据库性能

112. **MySQL主从复制的原理？** (参考链接)

- **核心组件**：
  - Master的binlog dump线程
  - Slave的I/O线程
  - Slave的SQL线程
- **复制过程**：
  - Master将变更记录到binlog
  - Slave的I/O线程读取Master的binlog

    - 将binlog事件写入relay log
    - Slave的SQL线程执行relay log中的事件

113. **主从复制延迟如何解决？** (参考链接)

- **原因分析**：
    - 网络延迟
    - 从库性能差
    - 主库并发写入量大
    - 从库有慢查询
- **解决方案**：
    - 优化网络环境
    - 提升从库硬件配置
    - 优化SQL查询
    - 考虑多线程复制
    - 减少主库写入压力

114. **什么是分库分表？** (参考链接)

- **定义**：将单一数据库/表拆分成多个数据库/表，以分散访问压力
- **分库**：按业务或数据特征将表分布在不同数据库
- **分表**：将单表数据按某种规则拆分到多个表
- **目的**：解决单库性能瓶颈、存储容量限制
- **挑战**：
    - 跨库事务处理
    - 跨表JOIN查询
    - 全局ID生成
    - 数据迁移

115. **水平分表和垂直分表的区别？** (参考链接)

- **水平分表**：
    - 按行拆分，不同行存储在不同表
    - 分片键选择关键
    - 保持表结构一致
    - 适合数据量大的场景
- **垂直分表**：
    - 按列拆分，不同列存储在不同表
    - 将热点字段和冷数据分离
    - 减少I/O，提高缓存效率

    - 适合字段多的场景

116. **分库分表后的ID如何生成？** (参考链接)

- **UUID**：全局唯一，但无序，索引效率低
- **雪花算法**：Twitter开源，64位ID，包含时间戳、机器ID、序列号
- **数据库分段**：
    - 每个库/表分配ID段
    - 例如：DB1: 1-10000, DB2: 10001-20000
- **Redis自增**：利用Redis的原子操作
- **专门的ID生成服务**：如Leaf、TinyID

117. **什么是读写分离？** (参考链接)

- **定义**：将数据库读操作和写操作分离到不同数据库实例
- **架构**：
    - 写库（主库）：处理INSERT、UPDATE、DELETE
    - 读库（从库）：处理SELECT查询
    - 通过主从复制同步数据
- **优势**：
    - 提高系统并发能力
    - 减轻主库压力
    - 提高查询性能
- **挑战**：
    - 主从延迟
    - 事务一致性
    - 读写分离策略

118. **数据库连接池的作用？** (参考链接)

- **资源复用**：
    - 预先创建连接，避免频繁创建销毁
    - 连接复用，提高性能
- **性能优化**：
    - 减少连接建立时间
    - 控制并发连接数
    - 提供连接管理功能
- **统一管理**：
    - 连接状态监控
    - 连接泄漏检测

    - 连接健康检查

119. **常用的数据库连接池有哪些？** (参考链接)

- **C3P0**：老牌连接池，配置复杂，性能一般
- **DBCP**：Apache开源，配置简单，性能中等
- **Druid**：阿里巴巴开源，功能丰富，监控能力强
- **HikariCP**：性能最优，配置简单，Spring Boot默认
- **Tomcat JDBC Pool**：Tomcat内置，性能较好

120. **什么是数据库锁？** (参考链接)

- **定义**：数据库用于管理对数据并发访问的机制
- **目的**：保证数据一致性和完整性
- **分类**：
    - **按粒度**：表锁、行锁、页锁
    - **按类型**：共享锁（S锁）、排他锁（X锁）
    - **按实现**：乐观锁、悲观锁
- **作用**：防止多个事务同时修改同一数据

121. **MySQL的锁类型？** (参考链接)

- **表级锁**：
    - 开销小，加锁快
    - 发生锁冲突概率高
    - 适合查询和更新少量数据
- **行级锁**：
    - 开销大，加锁慢
    - 发生锁冲突概率低
    - 适合高并发场景
- **意向锁**：
    - IS锁（意向共享锁）
    - IX锁（意向排他锁）

122. **什么是死锁？如何避免？** (参考链接)

- **定义**：多个事务相互等待对方释放锁，形成循环等待
- **产生条件**：
    - 互斥条件
    - 请求和保持条件

    - 不剥夺条件
    - 循环等待条件
- **避免方法**：
    - 按固定顺序访问资源
    - 使用超时机制
    - 降低事务隔离级别
    - 避免长事务

123. **如何排查数据库性能问题？** (参考链接)

- **性能监控**：
    - 使用SHOW PROCESSLIST查看当前连接
    - 使用performance_schema监控性能
    - 分析慢查询日志
- **SQL优化**：
    - 使用EXPLAIN分析执行计划
    - 检查索引使用情况
    - 优化JOIN、子查询
- **系统层面**：
    - 检查硬件资源使用
    - 检查MySQL配置参数
    - 监控网络状况

124. **MySQL的执行计划怎么看？** (参考链接)

- **id**：查询序号，越大越先执行，相同则从上往下
- **select_type**：查询类型（SIMPLE、PRIMARY、SUBQUERY等）
- **table**：查询的表名
- **type**：访问类型（ALL、index、range、ref、eq_ref、const、system）
- **possible_keys**：可能使用的索引
- **key**：实际使用的索引
- **rows**：预计扫描行数
- **Extra**：额外信息（Using Index、Using Where等）

125. **什么是覆盖索引？** (参考链接)

- **定义**：查询的字段全部包含在索引中的索引
- **优势**：
    - 无需回表查询

    - 减少I/O操作
    - 提高查询性能
- **示例**：索引(a,b,c)，查询SELECT a,b FROM table WHERE a=? AND b=?
- **注意事项**：
    - 避免SELECT *
    - 合理设计联合索引
    - 注意索引覆盖的局限性

6. Redis（20题）

126. **Redis的数据类型有哪些?** (参考链接)

- **String（字符串）**：最基本的数据类型，可以存储字符串、整数、浮点数
- **Hash（哈希）**：键值对集合，适合存储对象
- **List（列表）**：有序字符串列表，按插入顺序排序
- **Set（集合）**：无序字符串集合，元素唯一，支持集合操作
- **Sorted Set（有序集合）**：每个元素关联分数，按分数排序
- **其他类型**：
    - **Bitmaps**：基于String，提供位操作
    - **HyperLogLog**：基数统计算法
    - **Geo**：地理位置信息
    - **Streams**：消息流，支持消息队列
- **建议配图**：Redis数据类型对比表格，展示各类型的特性和适用场景

127. **Redis的持久化方式?** (参考链接)

- **RDB（Redis Database）**：
    - 在指定时间间隔内将内存数据快照写入磁盘
    - 通过fork子进程执行，父进程继续处理请求
    - 文件紧凑，适合备份和灾难恢复
- **AOF（Append Only File）**：
    - 记录每个写操作命令到日志文件
    - 通过重放命令恢复数据
    - 数据完整性更高，但文件体积较大
- **混合持久化**：Redis 4.0后支持，结合RDB和AOF优势

128. **RDB和AOF的区别?** (参考链接)

- **数据安全性**：AOF数据更安全，丢失数据更少；RDB可能丢失最后一次快照后的数据

- **性能影响**：RDB恢复速度快，AOF恢复速度相对较慢
- **文件大小**：RDB文件更小，AOF文件更大
- **运行效率**：RDB写入效率高，AOF根据策略不同效率各异
- **适用场景**：RDB适合备份和灾难恢复，AOF适合数据安全要求高的场景

129. **Redis的过期键删除策略？** (参考链接)

- **惰性删除**：
  - 获取key时检查是否过期
  - 过期则删除并返回null
  - 节约CPU资源，但内存回收不及时
- **定期删除**：
  - 定期随机抽取部分过期key删除
  - 平衡内存和CPU资源
  - 需要控制扫描频率
- **策略组合**：实际采用惰性删除+定期删除组合策略

130. **Redis的内存淘汰策略？** (参考链接)

- **noeviction**：默认策略，内存不足时返回错误
- **allkeys-lru**：移除最少使用的key（推荐）
- **volatile-lru**：移除设置了过期时间的最少使用key
- **allkeys-random**：随机移除key
- **volatile-random**：随机移除设置了过期时间的key
- **volatile-ttl**：移除即将过期的key

131. **Redis的主从复制原理？** (参考链接)

- **建立连接**：从节点向主节点发起SYNC命令
- **数据同步**：
  - 主节点执行BGSAVE生成RDB快照
  - 将快照文件发送给从节点
  - 从节点清空数据并加载快照
- **增量同步**：
  - 主节点将后续写命令发送给从节点
  - 从节点执行相同命令保持数据一致
- **心跳检测**：PING命令维持主从连接

132. **Redis集群的实现原理？** (参考链接)

- **分片机制**：
  - 数据按哈希槽（hash slot）分布
  - 共16384个槽位，每个key根据CRC16算法分配到槽
- **节点通信**：
  - 使用Gossip协议传播集群状态
  - 每秒发送ping/pong消息
- **故障转移**：
  - 主节点故障时，从节点晋升为主节点
  - 需要多数master节点同意
- **客户端路由**：
  - 通过ASK/MOVED重定向
  - 客户端维护槽位映射表
- **建议配图**：Redis集群拓扑图，展示节点间的关系和槽位分配

133. **Redis和Memcached的区别？** (参考链接)

- **数据类型**：Redis支持多种数据类型，Memcached只支持字符串
- **持久化**：Redis支持持久化，Memcached不支持
- **数据淘汰**：Redis有丰富的淘汰策略，Memcached简单LRU
- **内存管理**：Redis使用内存池，Memcached使用slab分配
- **网络模型**：Redis单线程，Memcached多线程
- **主从复制**：Redis支持，Memcached不支持

134. **什么是缓存穿透？如何解决？** (参考链接)

- **定义**：查询不存在的数据，缓存和数据库都没有，每次查询都打到数据库
- **危害**：大量无效请求冲击数据库
- **解决方案**：
  - **布隆过滤器**：在缓存前加一层过滤
  - **缓存空值**：将空结果也缓存一段时间
  - **参数校验**：严格校验请求参数
  - **实时监控**：监控异常请求

135. **什么是缓存击穿？如何解决？** (参考链接)

- **定义**：热点数据在缓存过期瞬间，大量请求涌入数据库
- **与穿透区别**：击穿针对单个热点key，穿透针对不存在的key
- **解决方案**：
  - **互斥锁**：查询数据库时加锁，其他请求等待

    - **永不过期**：更新数据时异步更新缓存
    - **提前刷新**：接近过期时间时主动刷新
    - **热点数据锁定**：对热点数据使用分布式锁

136. **什么是缓存雪崩？如何解决？** (参考链接)

- **定义**：大量缓存key同时过期，导致数据库压力过大
- **解决方案**：
    - **过期时间随机化**：在基础过期时间上加随机值
    - **缓存预热**：系统启动时提前加载热点数据
    - **多级缓存**：使用本地缓存+Redis缓存
    - **服务降级**：缓存失效时返回默认值
    - **熔断机制**：数据库压力过大时熔断部分请求

137. **Redis分布式锁的实现？** (参考链接)

- **基本实现**：
    - 使用SET key value NX EX seconds命令
    - NX确保key不存在时才设置
    - EX设置过期时间防止死锁
- **安全性考虑**：
    - 设置合理的过期时间
    - 使用Lua脚本保证原子性
    - 释放锁时验证所有权
- **Redlock算法**：在多个Redis实例上实现锁，提高可靠性

138. **Redis的事务？** (参考链接)

- **实现方式**：
    - MULTI：开启事务
    - EXEC：执行事务
    - DISCARD：取消事务
    - WATCH：监视key变化
- **特点**：
    - 保证命令的原子性执行
    - 不支持回滚
    - 不支持事务嵌套
- **局限性**：
    - 不能回滚，失败命令后的命令仍会执行

    - 不支持复杂的事务逻辑

139. **Redis的管道技术？** (参考链接)

- **定义**：客户端将多个命令打包一次性发送给Redis服务器
- **优势**：
    - 减少网络往返时间
    - 提高批量操作性能
    - 降低网络开销
- **使用场景**：
    - 批量插入数据
    - 批量查询操作
    - 减少网络延迟影响

140. **Redis的发布订阅？** (参考链接)

- **基本概念**：
    - PUBLISH：发布消息到频道
    - SUBSCRIBE：订阅频道接收消息
    - UNSUBSCRIBE：取消订阅
- **模式订阅**：使用PSUBSCRIBE订阅模式
- **应用场景**：
    - 实时消息推送
    - 系统解耦
    - 事件通知
- **局限性**：
    - 消息不持久化
    - 消费者离线时消息丢失

141. **Redis哨兵模式？** (参考链接)

- **作用**：
    - 监控主从节点健康状态
    - 自动故障转移
    - 通知客户端新的主节点
- **架构组成**：
    - 哨兵节点：监控Redis节点
    - 主从节点：提供数据服务
- **故障转移流程**：
    - 检测主节点故障

    - 选举新的主节点
    - 更新配置信息
    - 通知客户端

142. **Redis集群的槽位分配？** (参考链接)#slots

- **槽位总数**：16384个哈希槽（0-16383）
- **分配方式**：
    - 每个key通过CRC16(key) % 16384确定槽位
    - 每个主节点负责一部分槽位
- **槽位迁移**：
    - 扩容时重新分配槽位
    - 迁移过程支持读写操作
    - 保证数据一致性

143. **如何保证Redis和MySQL数据一致性？** (参考链接)

- **Cache-Aside模式**：
    - 读：先查缓存，缓存没有则查数据库并更新缓存
    - 写：更新数据库，删除缓存
- **双写一致性**：
    - 先更新数据库，再更新缓存
    - 使用分布式锁保证原子性
- **异步双写**：
    - 使用消息队列异步同步数据
    - 最终一致性保证

144. **Redis的性能优化？** (参考链接)

- **配置优化**：
    - 调整maxmemory限制内存使用
    - 选择合适的内存淘汰策略
    - 启用压缩列表优化小对象存储
- **数据结构优化**：
    - 合理选择数据类型
    - 设置合理的过期时间
    - 使用Pipeline批量操作
- **网络优化**：
    - 使用连接池
    - 减少网络往返次数

- 启用TCP连接复用

145. **Redis的pipeline和普通命令的区别?**

- **执行方式**：
  - 普通命令：每次发送单个命令，等待响应后再发下一个
  - Pipeline：将多个命令打包一次性发送，批量接收响应
- **网络开销**：
  - 普通命令：每次网络往返都有RTT延迟
  - Pipeline：大幅减少网络往返次数
- **性能差异**：
  - Pipeline性能远高于普通命令，尤其在批量操作时
- **原子性**：
  - 两者都不保证跨命令的原子性
  - Pipeline只是网络优化，不提供事务保证

7. 框架相关（25题）

146. **Spring的两大特性是什么?** (参考链接)

- **IoC（控制反转）**：
  - 将对象的创建和管理交给Spring容器
  - 降低代码耦合度
  - 通过依赖注入实现
- **AOP（面向切面编程）**：
  - 将横切关注点（如日志、事务）与业务逻辑分离
  - 提高代码复用性
  - 通过动态代理实现

147. **Spring的IoC实现原理?** (参考链接)

- **BeanFactory**：IoC容器的基础实现
- **ApplicationContext**：BeanFactory的子接口，提供更多功能
- **实现过程**：
  - 读取配置元数据（XML/注解）
  - 解析并注册BeanDefinition
  - 实例化Bean并进行依赖注入
  - 管理Bean生命周期
- **核心组件**：BeanDefinition、BeanFactory、BeanPostProcessor

148. **Spring的AOP实现原理？** (参考链接)

- **动态代理**：
  - JDK动态代理：基于接口实现
  - CGLIB代理：基于子类继承
- **实现机制**：
  - 在目标方法前后织入切面逻辑
  - 通过ProxyFactory创建代理对象
  - 支持前置、后置、环绕、异常通知
- **应用场景**：事务管理、日志记录、安全控制

149. **Spring Bean的生命周期？** (参考链接)

- **实例化**：通过反射创建Bean实例
- **属性填充**：注入依赖的其他Bean
- **Aware回调**：设置BeanName、BeanFactory等
- **BeanPostProcessor前置处理**：postProcessBeforeInitialization
- **初始化**：执行InitializingBean接口或@PostConstruct方法
- **BeanPostProcessor后置处理**：postProcessAfterInitialization
- **使用**：Bean处于可用状态
- **销毁**：执行DisposableBean接口或@PreDestroy方法

150. **Spring Bean的作用域？** (参考链接)

- **singleton**：默认作用域，整个Spring容器中只有一个实例
- **prototype**：每次请求创建新实例
- **request**：Web环境下，每个HTTP请求创建一个实例
- **session**：Web环境下，每个HTTP会话创建一个实例
- **application**：Web环境下，整个ServletContext生命周期内
- **websocket**：WebSocket会话内

151. **Spring的循环依赖如何解决？** (参考链接)

- **三级缓存**：
  - singletonObjects：单例对象缓存池
  - earlySingletonObjects：早期单例对象缓存
  - singletonFactories：单例工厂缓存
- **解决过程**：
  - 实例化A，放入singletonFactories
  - 发现A依赖B，实例化B

    - B依赖A，从singletonFactories获取A的工厂创建早期引用
    - B创建完成，注入A
    - A创建完成
- **限制**：只能解决setter注入的循环依赖，构造器注入无法解决

152. **Spring事务的传播机制？** (参考链接)

- **REQUIRED**：默认值，有事务则加入，无事务则新建
- **SUPPORTS**：有事务则加入，无事务则非事务执行
- **MANDATORY**：必须有事务，否则抛异常
- **REQUIRES_NEW**：新建事务，挂起当前事务
- **NOT_SUPPORTED**：非事务执行，挂起当前事务
- **NEVER**：非事务执行，有事务则抛异常
- **NESTED**：嵌套事务，当前有事务则创建保存点

153. **Spring Boot的自动配置原理？** (参考链接)

- **@EnableAutoConfiguration**：启用自动配置
- **SpringFactoriesLoader**：加载META-INF/spring.factories中配置的自动配置类
- **@Conditional**：条件注解，根据条件决定是否创建Bean
- **配置类**：通过@Configuration注解的配置类创建默认Bean
- **属性绑定**：通过@ConfigurationProperties绑定配置属性

154. **Spring Boot启动过程？** (参考链接)

- **创建SpringApplication**：
    - 推断应用类型
    - 加载ApplicationContextInitializer
    - 加载ApplicationListener
- **运行run方法**：
    - 启动计时
    - 监听器启动
    - 环境准备
    - 上下文创建
    - 环境设置
    - 后处理器应用
    - Bean加载
    - 启动完成

155. **Spring Cloud的核心组件？** (参考链接)

- **服务注册中心**：Eureka、Consul、Nacos
- **负载均衡**：Ribbon、Spring Cloud LoadBalancer
- **服务调用**：Feign、OpenFeign
- **熔断器**：Hystrix、Sentinel
- **网关**：Zuul、Spring Cloud Gateway
- **配置中心**：Spring Cloud Config、Nacos
- **服务链路追踪**：Zipkin、Sleuth

156. **什么是服务注册和发现？** (参考链接)

- **服务注册**：
  - 服务启动时向注册中心注册自身信息
  - 包括IP、端口、服务名等
- **服务发现**：
  - 客户端从注册中心获取服务列表
  - 实现服务调用
- **实现方式**：
  - 客户端发现模式：客户端直接访问注册中心
  - 服务端发现模式：通过负载均衡器访问

157. **什么是熔断器？** (参考链接)

- **定义**：
  - 当服务调用失败率达到阈值时，熔断器打开
  - 阻止后续请求调用该服务
  - 一段时间后尝试恢复
- **状态**：
  - 关闭：正常调用
  - 打开：拒绝请求
  - 半开：尝试恢复
- **作用**：
  - 防止服务雪崩
  - 提高系统稳定性

158. **什么是API网关？** (参考链接)

- **定义**：
  - 统一入口，处理所有客户端请求
  - 路由到后端微服务

- **功能**：
  - 路由转发
  - 负载均衡
  - 身份认证
  - 权限控制
  - 限流熔断
  - 日志监控
- **实现**：
  - Spring Cloud Gateway
  - Zuul
  - Kong

159. **MyBatis的#{}和${}的区别？** (参考链接)

- **#{}**：
  - 预编译处理，防止SQL注入
  - 会将参数加上引号
  - 适用于大多数场景
- **${}**：
  - 直接替换，不做预编译
  - 有SQL注入风险
  - 适用于动态表名、列名等场景

160. **MyBatis的一级缓存和二级缓存？** (参考链接)

- **一级缓存**：
  - SqlSession级别缓存
  - 默认开启
  - 同一SqlSession内有效
  - 增删改操作会清空缓存
- **二级缓存**：
  - SqlSessionFactory级别缓存
  - 需要手动开启
  - 跨SqlSession有效
  - 需要实体类实现Serializable接口

161. **如何防止SQL注入？** (参考链接)

- **使用预编译语句**：PreparedStatement代替Statement
- **参数绑定**：使用占位符而非字符串拼接

- **输入验证**：
  - 验证数据类型
  - 长度限制
  - 特殊字符过滤
- **ORM框架**：使用Hibernate、MyBatis等框架
- **权限控制**：最小权限原则
- **Web框架防护**：使用框架内置防护机制

162. **Spring MVC的执行流程？** (参考链接)

- **请求到达DispatcherServlet**
- **HandlerMapping查找处理器**
- **HandlerAdapter执行处理器**
- **执行拦截器preHandle方法**
- **执行Controller方法**
- **执行拦截器postHandle方法**
- **视图解析器解析视图**
- **渲染视图**
- **执行拦截器afterCompletion方法**

163. **什么是拦截器？** (参考链接)

- **定义**：
  - 在请求处理前后进行拦截处理
  - 实现HandlerInterceptor接口
- **执行时机**：
  - preHandle：Controller方法执行前
  - postHandle：Controller方法执行后，视图渲染前
  - afterCompletion：请求完成后
- **应用场景**：
  - 登录验证
  - 权限检查
  - 日志记录
  - 性能监控

164. **Spring中的设计模式？** (参考链接)

- **工厂模式**：BeanFactory、ApplicationContext
- **单例模式**：默认Bean作用域

- **代理模式**：AOP实现
- **模板方法模式**：JdbcTemplate、RestTemplate
- **策略模式**：ResourceLoader
- **观察者模式**：ApplicationListener
- **装饰器模式**：BeanWrapper
- **适配器模式**：HandlerAdapter

165. **Spring Boot的优点？** (参考链接)

- **快速开发**：
  - 约定优于配置
  - 内嵌Web服务器
  - 自动配置
- **简化部署**：
  - 打包为可执行jar
  - 无需外部Web容器
- **生态完善**：
  - 丰富的starter依赖
  - 与Spring生态无缝集成
- **生产就绪**：
  - 健康检查
  - 指标监控
  - 外部化配置

166. **Spring Cloud Gateway和Zuul的区别？** (参考链接)

- **性能**：Gateway基于WebFlux，异步非阻塞，性能更高
- **架构**：Gateway使用Netty，Zuul 1.x基于Servlet，Zuul 2.x也基于Netty
- **功能**：Gateway功能更丰富，路由规则更灵活
- **Filter**：Gateway使用WebFilter，Zuul使用ZuulFilter
- **维护状态**：Gateway是Spring官方维护，Zuul已停止维护

167. **Feign的实现原理？** (参考链接)

- **动态代理**：
  - 通过JDK动态代理创建接口实现
  - 拦截方法调用
- **注解解析**：
  - 解析@RequestMapping等注解

    - 构建HTTP请求
- **负载均衡**：
    - 集成Ribbon实现客户端负载均衡
- **执行请求**：
    - 通过HTTP客户端发送请求
    - 解析响应结果

168. **什么是声明式事务？** (参考链接)

- **定义**：
    - 通过注解或XML配置实现事务管理
    - 无需在代码中显式控制事务
- **实现方式**：
    - @Transactional注解
    - 基于AOP的事务增强
- **优势**：
    - 代码侵入性低
    - 业务逻辑与事务管理分离
    - 统一事务配置

169. **Spring Security的作用？** (参考链接)

- **认证**：
    - 用户身份验证
    - 支持多种认证方式
- **授权**：
    - 访问权限控制
    - 细粒度权限管理
- **防护**：
    - CSRF防护
    - XSS防护
    - Session固定攻击防护
- **集成**：
    - 与Spring框架深度集成
    - 支持OAuth2、JWT等协议

170. **什么是OAuth2.0？** (参考链接)

- **定义**：
    - 开放授权协议

    - 允许第三方应用获取有限访问权限
- **角色**：
    - 资源所有者：用户
    - 客户端：第三方应用
    - 资源服务器：提供资源
    - 授权服务器：发放访问令牌
- **授权模式**：
    - 授权码模式
    - 简化模式
    - 密码模式
    - 客户端凭证模式

8. 分布式系统（20题）

171. **什么是CAP定理？** (参考链接)

- **定义**：在一个分布式系统中，一致性（Consistency）、可用性（Availability）、分区容错性（Partition tolerance）三者不能同时满足
- **三个要素**：
    - **一致性（C）**：所有节点在同一时刻具有相同的数据
    - **可用性（A）**：每次请求都能收到非错响应，但不保证数据最新
    - **分区容错性（P）**：系统能继续运行即使网络分区出现
- **选择原则**：
    - CA：放弃分区容错性，适用于单机系统
    - CP：放弃可用性，保证一致性和分区容错性
    - AP：放弃一致性，保证可用性和分区容错性
- **建议配图**：CAP三角形示意图，展示三个顶点及三种权衡选择的关系

172. **什么是BASE理论？** (参考链接)

- **定义**：
    - **Basically Available（基本可用）**：系统在出现故障时仍能提供基本服务
    - **Soft State（软状态）**：允许系统中的数据存在中间状态
    - **Eventually Consistent（最终一致性）**：经过一定时间后系统数据达到一致
- **与ACID对比**：BASE是对CAP中AP的延伸，强调最终一致性而非强一致性
- **应用场景**：大规模分布式系统，如互联网应用

173. **分布式ID生成方案？** (参考链接)

- **UUID**：
  - 优点：全局唯一，生成简单
  - 缺点：无序，占用空间大，索引效率低
- **数据库自增ID**：
  - 优点：单调递增，有序
  - 缺点：单点故障，性能瓶颈
- **雪花算法（Snowflake）**：
  - 64位ID：1位符号位+41位时间戳+10位机器ID+12位序列号
  - 优点：全局唯一，趋势递增，性能好
  - 缺点：依赖系统时钟
- **Redis自增**：利用INCR原子操作
- **专门ID服务**：如Twitter的Snowflake、美团的Leaf

174. **分布式事务解决方案？** (参考链接)

- **2PC（两阶段提交）**：
  - 准备阶段：协调者询问参与者是否可以提交
  - 提交阶段：根据参与者回复决定提交或回滚
  - 缺点：同步阻塞，单点故障，数据不一致
- **3PC（三阶段提交）**：
  - 增加预提交阶段，减少阻塞
  - 仍存在数据不一致问题
- **TCC（Try-Confirm-Cancel）**：
  - Try：预留资源
  - Confirm：确认操作
  - Cancel：取消操作
  - 优点：无长期锁，高性能
- **SAGA**：将事务分解为一系列本地事务，每个有补偿操作
- **本地消息表**：通过消息队列保证最终一致性
- **最大努力通知**：反复通知直到成功

175. **什么是分布式锁？** (参考链接)

- **定义**：在分布式系统中，控制多个节点对共享资源的访问
- **实现方式**：
  - **基于数据库**：利用数据库唯一索引
  - **基于Redis**：使用SET NX EX命令
  - **基于Zookeeper**：利用临时顺序节点
- **特性要求**：

    - 互斥性：同一时刻只有一个客户端能持有锁
    - 容错性：部分节点故障不影响锁功能
    - 释放性：锁最终会被释放
- **应用场景**：
    - 资源抢占
    - 防止重复提交
    - 分布式任务调度

176. **Zookeeper的选举机制？** (参考链接)

- **选举算法**：ZAB协议（Zookeeper Atomic Broadcast）
- **选举过程**：
    - 每个节点都有ZXID（事务ID）和MYID（节点ID）
    - 节点启动时投票给自己，格式为(MYID, ZXID)
    - 节点间交换投票信息，选择ZXID最大者
    - 若ZXID相同，则选择MYID最大者
    - 当超过半数节点投票给同一节点时，该节点成为Leader
- **选举状态**：
    - LOOKING：选举状态
    - LEADING：领导状态
    - FOLLOWING：跟随状态
    - OBSERVING：观察状态

177. **Zookeeper的作用？** (参考链接)

- **配置管理**：
    - 集中管理配置信息
    - 配置变更时实时通知客户端
- **命名服务**：
    - 提供全局唯一路径标识
    - 服务发现和注册
- **分布式锁**：
    - 利用临时顺序节点实现排他锁
    - 实现分布式同步
- **集群管理**：
    - 监控节点状态
    - 实现主节点选举
- **分布式队列**：
    - 同步队列

    - 先进先出队列

178. **什么是负载均衡？** (参考链接)

- **定义**：将负载（工作量）分配到多个操作单元上的机制
- **作用**：
    - 提高系统吞吐量
    - 避免单点过载
    - 提高系统可用性
    - 实现横向扩展
- **分类**：
    - **硬件负载均衡**：F5、A10等专用设备
    - **软件负载均衡**：Nginx、HAProxy、LVS
    - **DNS负载均衡**：通过DNS解析实现
    - **客户端负载均衡**：Ribbon等在客户端实现

179. **负载均衡算法有哪些？** (参考链接)

- **轮询（Round Robin）**：
    - 按顺序分配请求
    - 适用于服务器性能相近场景
- **加权轮询（Weighted RR）**：
    - 根据服务器性能分配权重
    - 性能好的服务器分配更多请求
- **最少连接（Least Connections）**：
    - 将请求分配给连接数最少的服务器
    - 适用于长连接场景
- **加权最少连接**：
    - 结合权重和连接数考虑
- **随机算法**：
    - 随机选择服务器
    - 简单高效
- **IP哈希**：
    - 根据客户端IP计算哈希值
    - 同一IP总是路由到同一服务器

180. **什么是服务熔断？** (参考链接)

- **定义**：

    - 当下游服务故障率超过阈值时，熔断器打开
    - 暂时切断对该服务的调用
    - 避免故障扩散
- **三种状态**：
    - **关闭状态**：正常调用，记录失败率
    - **打开状态**：直接拒绝请求，快速失败
    - **半开状态**：尝试恢复，允许部分请求通过
- **实现机制**：
    - 计数器模式
    - 时间窗口模式
- **作用**：
    - 防止服务雪崩
    - 提高系统稳定性

181. **什么是服务降级？** (参考链接)

- **定义**：
    - 在系统资源紧张时，暂时关闭某些非核心功能
    - 保证核心服务正常运行
- **触发条件**：
    - 服务器压力过大
    - 某些服务异常
    - 资源不够用
- **实现方式**：
    - 页面降级：返回友好提示
    - 延迟服务：异步处理
    - 写降级：只读不写
    - 读降级：缓存降级
- **降级策略**：
    - 自动降级
    - 人工降级

182. **什么是服务限流？** (参考链接)

- **定义**：
    - 控制单位时间内系统处理的请求数量
    - 防止系统被突发流量冲垮
- **限流维度**：
    - **时间维度**：每秒/分钟/小时请求数

    - **资源维度**：CPU、内存、数据库连接数
    - **用户维度**：单用户请求频率
    - **接口维度**：单接口访问频率
- **作用**：
    - 保护系统稳定性
    - 保证服务质量
    - 防止恶意攻击

183. **限流算法有哪些？** (参考链接)

- **计数器算法**：
    - 固定时间窗口内统计请求数
    - 超过阈值则拒绝
    - 存在临界问题
- **滑动窗口算法**：
    - 将时间窗口划分为多个小窗口
    - 精确控制流量
    - 解决计数器算法临界问题
- **漏桶算法**：
    - 请求进入固定容量的漏桶
    - 以恒定速率流出
    - 平滑流量，控制输出速率
- **令牌桶算法**：
    - 按固定速率生成令牌
    - 请求需要令牌才能处理
    - 支持突发流量

184. **什么是分布式一致性？** (参考链接)

- **定义**：分布式系统中多个副本之间的数据一致性程度
- **一致性级别**：
    - **强一致性**：更新完成后所有读操作都返回新值
    - **弱一致性**：更新后不保证立即同步
    - **最终一致性**：经过一定时间后达到一致
- **一致性模型**：
    - 线性一致性
    - 顺序一致性
    - 因果一致性
    - PRAM一致性

- **实现机制**：
  - 两阶段提交
  - 多数派协议
  - 向量时钟

185. **Raft算法原理？** (参考链接)

- **核心思想**：
  - 选举：选出Leader节点
  - 日志复制：Leader向Follower同步日志
  - 安全性：保证状态机安全
- **节点状态**：
  - Leader：处理所有客户端请求
  - Follower：被动接收Leader请求
  - Candidate：参与Leader选举
- **任期（Term）**：
  - 单调递增的整数
  - 每次选举开始新任期
  - 各节点维护当前任期
- **选举流程**：
  - Follower超时转为Candidate
  - 发起选举投票
  - 获得多数票成为Leader
- **建议配图**：Raft状态转换图，展示Follower→Candidate→Leader的状态变迁过程和任期概念

186. **Paxos算法原理？** (参考链接)

- **定义**：分布式系统中解决一致性问题的经典算法
- **角色**：
  - Proposer：提出提案
  - Acceptor：批准提案
  - Learner：学习已批准的提案
- **两个阶段**：
  - **准备阶段**：Proposer向多数Acceptor发送准备请求
  - **接受阶段**：Acceptor接受提案或拒绝
- **约束条件**：
  - P1：Acceptor必须接受第一个收到的提案
  - P2：只有被多数派批准的提案才能被学习
- **特点**：

    - 容忍网络延迟和分区
    - 保证一致性
    - 但难以理解和实现

187. **分布式系统中的时钟同步？** (参考链接)

- **问题背景**：
    - 分布式系统中各节点时钟不完全同步
    - 影响事件顺序判断
- **解决方案**：
    - **NTP（Network Time Protocol）**：
        - 通过网络同步时钟
        - 精度毫秒级
    - **PTP（Precision Time Protocol）**：
        - 精度纳秒级
        - 适用于高精度场景
- **逻辑时钟**：
    - Lamport时钟：
        - 事件发生时递增
        - 发送消息时携带时间戳
        - 接收消息时更新时间戳
    - **向量时钟**：
        - 每个节点维护向量记录各节点时间
        - 更精确的因果关系判断

188. **什么是分布式追踪？** (参考链接)

- **定义**：
    - 跟踪请求在分布式系统中的完整调用链路
    - 用于性能分析和故障诊断
- **核心概念**：
    - **Trace**：一次完整的请求链路
    - **Span**：链路中的一个基本单元
    - **Trace ID**：标识一次完整请求
    - **Span ID**：标识一个调用单元
- **实现原理**：
    - 在服务调用时注入追踪信息
    - 收集并上报追踪数据
    - 可视化展示调用链路

- **常用框架**：
  - Zipkin
  - Jaeger
  - SkyWalking
  - OpenTracing/OpenTelemetry标准

189. **如何保证分布式系统的数据一致性？** (参考链接)

- **强一致性方案**：
  - 2PC/3PC：保证事务一致性
  - Quorum：多数派协议
  - Raft/Paxos：共识算法
- **最终一致性方案**：
  - 消息队列：异步同步数据
  - 版本号：检测数据冲突
  - 向量时钟：跟踪因果关系
- **实践策略**：
  - 根据业务需求选择一致性级别
  - 采用补偿机制处理不一致
  - 监控和修复数据不一致
  - 实现幂等性操作

190. **什么是分布式协调？** (参考链接)

- **定义**：
  - 在分布式系统中协调多个节点的行为
  - 解决分布式环境下的协作问题
- **协调任务**：
  - **Leader选举**：选择主节点
  - **配置管理**：集中管理配置信息
  - **分布式锁**：协调资源访问
  - **服务发现**：注册和发现服务
  - **命名服务**：提供全局唯一标识
  - **集群管理**：监控节点状态
- **实现工具**：
  - Zookeeper：提供一致性协调服务
  - Etcd：基于Raft协议
  - Consul：服务网格协调
- **应用场景**：

    - 分布式系统状态同步
    - 配置变更通知
    - 分布式任务调度

9. 系统设计（10题）

191. **如何设计一个短网址系统？** (参考链接)

- **核心功能**：
    - 长网址转短网址
    - 短网址还原长网址
- **关键技术**：
    - ID生成：雪花算法或数据库自增ID
    - 编码：Base62编码（0-9,a-z,A-Z）
    - 存储：Redis存储短码到长网址映射
- **架构设计**：
    - 负载均衡层：Nginx
    - Web服务层：处理短码生成和解析
    - 缓存层：Redis存储映射关系
    - 数据库：MySQL存储历史记录
- **优化策略**：
    - 预生成短码，避免实时编码
    - 设置缓存过期时间
    - 统计分析功能
- **建议配图**：短网址系统架构图，展示各组件间的关系和数据流向

192. **如何设计一个秒杀系统？** (参考链接)

- **核心挑战**：
    - 高并发：瞬时大量请求
    - 超卖：防止库存被多卖
    - 系统稳定性：保护核心系统
- **架构设计**：
    - 前端优化：静态化、CDN加速
    - 限流降级：控制流量
    - 缓存预热：提前加载商品信息
    - 库存预扣：Redis预扣库存
    - 异步处理：消息队列处理订单

- **关键技术**：
  - 页面静态化：减少服务端压力
  - CDN：静态资源加速
  - Redis：库存管理
  - 消息队列：异步处理订单
  - 分布式锁：防止超卖
- **保护措施**：
  - 用户限流：限制单用户购买数量
  - 防刷机制：验证码、IP限制
- **建议配图**：秒杀系统架构图，展示各层级的防护措施和数据流转

193. **如何设计一个消息队列?** (参考链接)

- **核心功能**：
  - 消息发送：Producer发送消息
  - 消息存储：Broker存储消息
  - 消息消费：Consumer消费消息
- **架构组件**：
  - Producer：消息生产者
  - Broker：消息代理服务器
  - Consumer：消息消费者
  - Topic/Queue：消息分类
- **关键技术**：
  - 持久化：确保消息不丢失
  - 高可用：主从复制、集群部署
  - 负载均衡：分区、副本机制
  - 消费确认：ACK机制
- **设计要点**：
  - 消息可靠性：至少一次、最多一次、恰好一次
  - 消息顺序性：保证有序消费
  - 消息积压：批量消费、多消费者

194. **如何设计一个分布式缓存?** (参考链接)

- **核心需求**：
  - 高性能：快速访问
  - 高可用：故障容错
  - 可扩展：水平扩展
- **架构设计**：

    - 客户端：缓存访问接口
    - 代理层：路由、负载均衡
    - 缓存节点：实际存储数据
    - 配置中心：集群配置管理
- **关键技术**：
    - 一致性哈希：数据分布算法
    - 分片机制：数据切分存储
    - 主从复制：数据冗余备份
    - 故障转移：自动故障检测
- **缓存策略**：
    - 缓存更新策略：Cache-Aside、Read-Through、Write-Through
    - 淘汰策略：LRU、LFU等
    - 缓存穿透、雪崩、击穿防护

195. **如何设计一个搜索引擎？** (参考链接)

- **核心功能**：
    - 爬虫：抓取网页内容
    - 索引：建立倒排索引
    - 搜索：提供检索服务
    - 排序：相关性排序
- **架构组件**：
    - 爬虫系统：网页抓取
    - 解析器：提取文本内容
    - 索引器：构建索引
    - 搜索器：提供搜索服务
    - 排序器：计算相关性
- **关键技术**：
    - 倒排索引：文档ID到关键词映射
    - TF-IDF：计算词频权重
    - 分词技术：中文分词
    - 全文检索：Elasticsearch等
- **优化策略**：
    - 索引分片：提高检索性能
    - 缓存机制：热点数据缓存
    - 相关性优化：学习排序算法

196. **如何设计一个分布式任务调度系统？** (参考链接)

- **核心功能**：
  - 任务定义：支持不同类型任务
  - 调度策略：定时、周期、依赖调度
  - 任务执行：分布式执行
  - 监控管理：任务状态监控
- **架构设计**：
  - 调度中心：任务调度核心
  - 执行节点：任务执行器
  - 存储层：任务配置存储
  - 监控中心：任务状态监控
- **关键技术**：
  - 分布式锁：防止重复执行
  - 故障转移：节点故障自动切换
  - 负载均衡：任务均匀分配
  - 任务分片：大任务拆分执行
- **调度策略**：
  - Cron表达式：定时调度
  - 依赖调度：任务间依赖关系
  - 失败重试：任务执行失败处理

197. **如何设计一个限流系统？** (参考链接)

- **限流维度**：
  - 接口限流：单接口访问频率
  - 用户限流：单用户访问频率
  - IP限流：单IP访问频率
  - 应用限流：单应用访问频率
- **架构设计**：
  - 限流规则配置：定义限流策略
  - 限流算法实现：计数器、滑窗、令牌桶等
  - 限流决策：判断是否允许访问
  - 限流执行：拒绝或排队处理
- **关键技术**：
  - Redis：存储限流计数
  - Lua脚本：保证操作原子性
  - 分布式协调：多节点限流同步
- **实现策略**：
  - 单机限流：基于内存计数

    - 分布式限流：基于Redis等存储
    - 网关限流：统一接入层限流

198. **如何设计一个用户权限系统？** (参考链接)

- **核心概念**：
    - 用户（User）：系统使用者
    - 角色（Role）：权限集合
    - 权限（Permission）：操作许可
    - 资源（Resource）：受保护对象
- **权限模型**：
    - RBAC（基于角色的访问控制）：
        - 用户-角色-权限关系
        - 灵活的权限分配
    - ACL（访问控制列表）：
        - 直接关联用户和权限
        - 精细的权限控制
- **架构设计**：
    - 认证模块：用户身份验证
    - 授权模块：权限验证
    - 管理模块：权限配置
    - 缓存模块：权限缓存
- **关键技术**：
    - JWT：无状态认证
    - OAuth2：授权协议
    - SSO：单点登录
    - 细粒度权限：基于URL、方法级别

199. **如何设计一个API网关？** (参考链接)

- **核心功能**：
    - 路由转发：请求路由到后端服务
    - 负载均衡：请求分发到多个实例
    - 认证授权：统一身份验证
    - 限流熔断：保护后端服务
    - 监控统计：请求监控和分析
- **架构设计**：
    - 路由层：请求路由和转发
    - 限流层：流量控制

    - 认证层：身份验证
    - 熔断层：服务保护
    - 监控层：指标收集
- **关键技术**：
    - 动态路由：服务发现机制
    - 负载均衡：多种算法支持
    - SSL卸载：HTTPS处理
    - 缓存机制：响应缓存
    - 服务编排：聚合多个服务
- **实现方案**：
    - Nginx+Lua：高性能网关
    - Spring Cloud Gateway：Java生态
    - Kong：基于OpenResty
    - Traefik：云原生网关

200. **如何设计一个实时消息推送系统？** (参考链接)

- **推送方式**：
    - WebSocket：全双工通信
    - Server-Sent Events：单向推送
    - 长轮询：HTTP长连接
- **架构设计**：
    - 连接管理层：维护客户端连接
    - 消息路由层：消息分发
    - 消息存储层：离线消息存储
    - 业务接入层：推送接口
- **关键技术**：
    - 连接管理：心跳检测、断线重连
    - 消息路由：按用户ID路由
    - 水平扩展：集群部署
    - 消息可靠性：ACK确认机制
- **性能优化**：
    - 连接复用：减少连接建立开销
    - 批量推送：提高推送效率
    - 消息压缩：减少网络传输
    - 离线推送：存储离线消息

10. 算法与数据结构（10题）

201. **手写快速排序算法** (参考链接)

```
public static void quickSort(int[] arr, int left, int right) {
    if (left >= right) return;

    int pivotIndex = partition(arr, left, right);
    quickSort(arr, left, pivotIndex - 1);
    quickSort(arr, pivotIndex + 1, right);
}

private static int partition(int[] arr, int left, int right) {
    int pivot = arr[right];
    int i = left - 1;

    for (int j = left; j < right; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr, i, j);
        }
    }
    swap(arr, i + 1, right);
    return i + 1;
}

private static void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
```

- **时间复杂度**：平均O(nlogn)，最坏O(n^2)
- **空间复杂度**：O(logn)
- **原理**：分治思想，选择基准元素，将数组分为小于和大于基准的两部分
- **建议配图**：快速排序执行过程图，展示分区过程和递归调用树结构

202. **手写二分查找算法** (参考链接)

```java
public static int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2; // 防止溢出
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}
```

- **前提条件**：数组必须有序
- **时间复杂度**：O(logn)
- **空间复杂度**：O(1)
- **应用场景**：查找特定元素、寻找边界值

203. **手写链表反转算法** (参考链接)

```java
public ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;

    while (curr != null) {
        ListNode nextTemp = curr.next;
        curr.next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}
```

- **时间复杂度**：O(n)
- **空间复杂度**：O(1)
- **思路**：逐个改变节点的指针方向
- **关键点**：保存下一个节点的引用，避免断链

204. **手写二叉树前序遍历（递归和非递归）**(参考链接)

- **递归实现**：

```
public void preorderTraversal(TreeNode root, List<Integer> result) {
    if (root == null) return;
    result.add(root.val);        // 访问根节点
    preorderTraversal(root.left, result);   // 遍历左子树
    preorderTraversal(root.right, result);  // 遍历右子树
}
```

- **非递归实现**：

```
public List<Integer> preorderTraversal(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Stack<TreeNode> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        result.add(node.val);

        if (node.right != null) stack.push(node.right); // 先压入右子树
        if (node.left != null) stack.push(node.left);   // 后压入左子树
    }
    return result;
}
```

- **时间复杂度**：O(n)
- **空间复杂度**：O(h)，h为树的高度

205. **手写LRU缓存算法** (参考链接)

```java
class LRUCache {
    class Node {
        int key, value;
        Node prev, next;
        Node(int k, int v) { key = k; value = v; }
    }

    class DoubleLinkedList {
        private Node head, tail;
        private int size;

        public DoubleLinkedList() {
            head = new Node(0, 0);
            tail = new Node(0, 0);
            head.next = tail;
            tail.prev = head;
            size = 0;
        }

        public void addFirst(Node node) {
            node.next = head.next;
            node.prev = head;
            head.next.prev = node;
            head.next = node;
            size++;
        }

        public void remove(Node node) {
            node.prev.next = node.next;
            node.next.prev = node.prev;
            size--;
        }

        public Node removeLast() {
            if (tail.prev == head) return null;
            Node last = tail.prev;
            remove(last);
            return last;
        }

        public int size() { return size; }
    }
```

```java
    private Map<Integer, Node> map = new HashMap<>();
    private DoubleLinkedList cache = new DoubleLinkedList();
    private int cap;

    public LRUCache(int capacity) {
        this.cap = capacity;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        int val = map.get(key).value;
        put(key, val); // 提升为最近使用
        return val;
    }

    public void put(int key, int value) {
        Node x = new Node(key, value);

        if (map.containsKey(key)) {
            cache.remove(map.get(key));
            cache.addFirst(x);
            map.put(key, x);
        } else {
            if (cache.size() == cap) {
                Node last = cache.removeLast();
                map.remove(last.key);
            }
            cache.addFirst(x);
            map.put(key, x);
        }
    }
}
```

- **时间复杂度**：get和put操作都是O(1)
- **空间复杂度**：O(capacity)
- **实现原理**：哈希表+双向链表

206. **如何判断链表是否有环？** (参考链接)

```java
public boolean hasCycle(ListNode head) {
    if (head == null || head.next == null) return false;

    ListNode slow = head, fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;           // 慢指针走一步
        fast = fast.next.next;      // 快指针走两步

        if (slow == fast) return true; // 相遇说明有环
    }
    return false;
}
```

- **算法**：Floyd判圈算法（快慢指针）
- **时间复杂度**：O(n)
- **空间复杂度**：O(1)
- **原理**：若有环，快指针一定会追上慢指针

207. **如何找到链表的倒数第K个节点？** (参考链接)

```java
public ListNode getKthFromEnd(ListNode head, int k) {
    ListNode fast = head, slow = head;

    // 快指针先走k步
    for (int i = 0; i < k; i++) {
        if (fast == null) return null; // k大于链表长度
        fast = fast.next;
    }

    // 一起移动，直到快指针到达末尾
    while (fast != null) {
        fast = fast.next;
        slow = slow.next;
    }

    return slow; // 此时慢指针指向倒数第k个节点
}
```

- **算法**：双指针技巧
- **时间复杂度**：O(n)

- **空间复杂度**：O(1)
- **原理**：两指针距离固定为k，当快指针到达末尾时，慢指针正好在倒数第k个位置

208. **两个栈实现一个队列** (参考链接)

```
class MyQueue {
    private Stack<Integer> inStack;
    private Stack<Integer> outStack;

    public MyQueue() {
        inStack = new Stack<>();
        outStack = new Stack<>();
    }

    public void push(int x) {
        inStack.push(x);
    }

    public int pop() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) {
                outStack.push(inStack.pop());
            }
        }
        return outStack.pop();
    }

    public int peek() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) {
                outStack.push(inStack.pop());
            }
        }
        return outStack.peek();
    }

    public boolean empty() {
        return inStack.isEmpty() && outStack.isEmpty();
    }
}
```

- **实现思路**：inStack负责入队，outStack负责出队

- **时间复杂度**：均摊O(1)
- **空间复杂度**：O(n)
- **关键点**：outStack为空时才将inStack元素转移过来

209. **如何判断一个数是不是质数？** (参考链接)

```
public boolean isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;

    // 只需检查到sqrt(n)
    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) {
            return false;
        }
    }
    return true;
}
```

- **优化思路**：
  - 只需检查到√n
  - 除2和3外，质数都在6的倍数两侧（6k±1）
- **时间复杂度**：O(√n)
- **空间复杂度**：O(1)

210. **动态规划：最长公共子序列** (参考链接)

```java
public int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    // dp[i][j] 表示text1[0..i-1]和text2[0..j-1]的LCS长度
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}
```

- **状态定义**：dp[i][j]表示两个字符串前缀的LCS长度
- **状态转移**：
  - 相等：dp[i][j] = dp[i-1][j-1] + 1
  - 不等：dp[i][j] = max(dp[i-1][j], dp[i][j-1])
- **时间复杂度**：O(mn)
- **空间复杂度**：O(mn)，可优化至O(min(m,n))

11. 网络与操作系统（10题）

211. **TCP和UDP的区别?** (参考链接)

- **连接性**：
  - TCP：面向连接，需要建立连接后传输数据
  - UDP：无连接，直接发送数据
- **可靠性**：
  - TCP：可靠传输，有确认、重传、拥塞控制机制
  - UDP：不可靠传输，不保证数据到达
- **有序性**：
  - TCP：保证数据按序到达
  - UDP：不保证数据顺序
- **传输效率**：
  - TCP：传输效率较低，开销大

    - UDP：传输效率高，开销小
- **适用场景**：
    - TCP：适用于可靠性要求高的场景，如HTTP、FTP、SMTP
    - UDP：适用于实时性要求高、可容忍少量丢包的场景，如视频直播、语音通话

212. **TCP三次握手的过程？** (参考链接)

- **第一次握手**：
    - 客户端发送SYN=1，seq=x到服务器
    - 客户端进入SYN_SENT状态
- **第二次握手**：
    - 服务器接收SYN，回复SYN=1，ACK=1，ack=x+1，seq=y
    - 服务器进入SYN_RCVD状态
- **第三次握手**：
    - 客户端发送ACK=1，ack=y+1，seq=x+1
    - 服务器接收后进入ESTABLISHED状态
    - 客户端发送完后也进入ESTABLISHED状态
- **建议配图**：TCP三次握手时序图，清晰展示客户端和服务器之间的消息交换过程和状态变化

213. **为什么需要三次握手而不是两次？** (参考链接)

- **防止已失效的连接请求突然传到服务器**：
    - 假设两次握手，客户端发出的第一个连接请求在网络中滞留后到达服务器
    - 服务器会建立连接，但客户端已忽略，造成资源浪费
- **确认双方的发送和接收能力**：
    - 两次握手只能确认一方的发送和接收能力
    - 三次握手能确认双方的发送和接收能力
- **同步初始序列号**：
    - 通过三次握手同步双方的初始序列号
    - 防止序列号混乱

214. **TCP四次挥手的过程？** (参考链接)

- **第一次挥手**：
    - 客户端发送FIN=1，seq=u到服务器，表示要关闭连接
    - 客户端进入FIN_WAIT_1状态
- **第二次挥手**：
    - 服务器接收FIN，发送ACK=1，ack=u+1，seq=v
    - 服务器进入CLOSE_WAIT状态
    - 客户端收到后进入FIN_WAIT_2状态

- **第三次挥手**：
  - 服务器发送FIN=1，ACK=1，seq=w，ack=u+1到客户端
  - 服务器进入LAST_ACK状态
- **第四次挥手**：
  - 客户端发送ACK=1，ack=w+1，seq=u+1到服务器
  - 客户端进入TIME_WAIT状态
  - 服务器收到后进入CLOSED状态

215. **什么是TIME_WAIT状态？** (参考链接)

- **定义**：
  - 客户端在收到服务器FIN后，发送ACK后进入TIME_WAIT状态
  - 持续2MSL（Maximum Segment Lifetime）时间
- **作用**：
  - 确保被动关闭方收到最后的ACK
  - 让网络中重复数据段消逝
- **问题**：
  - 大量TIME_WAIT会占用端口资源
  - 可能导致端口耗尽
- **优化**：
  - 调整内核参数tcp_tw_reuse、tcp_tw_recycle
  - 设置SO_REUSEADDR选项

216. **HTTP和HTTPS的区别？** (参考链接)

- **安全性**：
  - HTTP：明文传输，不安全
  - HTTPS：加密传输，安全
- **端口号**：
  - HTTP：80端口
  - HTTPS：443端口
- **协议**：
  - HTTP：应用层协议
  - HTTPS：HTTP + SSL/TLS
- **证书**：
  - HTTP：不需要证书
  - HTTPS：需要SSL证书
- **性能**：
  - HTTP：性能好，开销小

- HTTPS：性能稍差，有加密解密开销

217. **HTTP状态码有哪些？** (参考链接)

- **1xx信息性状态码**：
  - 100 Continue：继续
  - 101 Switching Protocols：切换协议
- **2xx成功状态码**：
  - 200 OK：成功
  - 201 Created：已创建
  - 204 No Content：无内容
- **3xx重定向状态码**：
  - 301 Moved Permanently：永久重定向
  - 302 Found：临时重定向
  - 304 Not Modified：未修改
- **4xx客户端错误状态码**：
  - 400 Bad Request：错误请求
  - 401 Unauthorized：未授权
  - 403 Forbidden：禁止访问
  - 404 Not Found：未找到
  - 405 Method Not Allowed：方法不允许
- **5xx服务器错误状态码**：
  - 500 Internal Server Error：服务器内部错误
  - 502 Bad Gateway：网关错误
  - 503 Service Unavailable：服务不可用
  - 504 Gateway Timeout：网关超时

218. **什么是HTTP长连接？** (参考链接)

- **定义**：
  - 在HTTP/1.1中，默认使用长连接
  - 一个TCP连接可以传输多个HTTP请求/响应
- **优势**：
  - 减少TCP连接建立和关闭的开销
  - 提高网络利用率
  - 减少服务器资源消耗
- **实现**：
  - HTTP请求头包含Connection: Keep-Alive
  - 服务器保持连接一段时间

    - 可设置最大请求数或超时时间
- **HTTP/2**：
    - 多路复用，一个连接并发处理多个请求
    - 进一步提高效率

219. **什么是CDN?** (参考链接)

- **定义**：
    - Content Delivery Network，内容分发网络
    - 将内容缓存到网络边缘节点
- **工作原理**：
    - 用户请求内容时，DNS解析到就近的CDN节点
    - CDN节点返回缓存内容
    - 若缓存未命中，向源站请求并缓存
- **优势**：
    - 提高访问速度
    - 减轻源站压力
    - 提高可用性
    - 降低带宽成本
- **关键技术**：
    - 缓存技术
    - 负载均衡
    - 内容路由
    - 内容管理

220. **Linux常用的命令有哪些?** (参考链接)

- **文件操作**：
    - ls：列出目录内容
    - cd：切换目录
    - pwd：显示当前路径
    - mkdir：创建目录
    - rm：删除文件/目录
    - cp：复制文件
    - mv：移动/重命名文件
- **系统监控**：
    - ps：查看进程
    - top：实时查看系统资源
    - free：查看内存使用

    - df：查看磁盘使用
    - du：查看目录大小
- **网络相关**：
    - netstat：网络连接状态
    - ifconfig：网络接口配置
    - ping：测试网络连通
    - telnet：远程连接
- **文本处理**：
    - cat：查看文件内容
    - grep：文本搜索
    - sed：流编辑器
    - awk：文本处理
- **权限管理**：
    - chmod：修改文件权限
    - chown：修改文件属主
    - su：切换用户

12. 设计模式（10题）

221. **单例模式的几种实现方式？** (参考链接)

- **饿汉式**：

```
public class Singleton {
    private static final Singleton INSTANCE = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() {
        return INSTANCE;
    }
}
```

- **懒汉式（线程安全）**：

```java
public class Singleton {
    private static Singleton instance;
    private Singleton() {}
    public static synchronized Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

- **双重检查锁定**：

```java
public class Singleton {
    private volatile static Singleton instance;
    private Singleton() {}
    public static Singleton getInstance() {
        if (instance == null) {
            synchronized (Singleton.class) {
                if (instance == null) {
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

- **静态内部类**：

```java
public class Singleton {
    private Singleton() {}
    private static class SingletonHolder {
        private static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() {
        return SingletonHolder.INSTANCE;
    }
}
```

- **枚举实现**：

```java
public enum Singleton {
    INSTANCE;
    public void doSomething() {
        // 业务方法
    }
}
```

222. **工厂模式的实现方式?** (参考链接)

- **简单工厂模式**:

```java
public class ShapeFactory {
    public static Shape getShape(String type) {
        if ("CIRCLE".equalsIgnoreCase(type)) {
            return new Circle();
        } else if ("RECTANGLE".equalsIgnoreCase(type)) {
            return new Rectangle();
        }
        return null;
    }
}
```

- **工厂方法模式**:

```java
public interface ShapeFactory {
    Shape createShape();
}
public class CircleFactory implements ShapeFactory {
    public Shape createShape() {
        return new Circle();
    }
}
```

- **抽象工厂模式**:

```java
public interface AbstractFactory {
    Shape createShape();
    Color createColor();
}
```

223. **观察者模式的应用场景？** (参考链接)

- **定义**：
  - 一个对象状态改变时，所有依赖它的对象都会收到通知
  - 实现一对多的依赖关系
- **应用场景**：
  - 事件驱动系统：GUI事件处理
  - 消息订阅：发布-订阅模式
  - MVC架构：Model和View的联动
  - Spring事件机制：ApplicationEvent
  - 股票价格变动通知
- **实现**：

```java
public interface Observer {
    void update(String message);
}
public class Subject {
    private List<Observer> observers = new ArrayList<>();
    public void attach(Observer observer) {
        observers.add(observer);
    }
    public void notifyObservers(String message) {
        for (Observer observer : observers) {
            observer.update(message);
        }
    }
}
```

224. **装饰器模式的实现？** (参考链接)

- **定义**：
  - 动态地给对象添加功能
  - 是继承的替代方案
- **作用**：
  - 动态扩展对象功能
  - 避免类爆炸
  - 符合开闭原则
- **应用场景**：
  - Java I/O流：BufferedReader、BufferedWriter
  - Spring Security：过滤器链

    - Servlet：Filter过滤器
- **实现**：

```
public interface Component {
    void operation();
}
public class ConcreteComponent implements Component {
    public void operation() {
        System.out.println("具体组件操作");
    }
}
public abstract class Decorator implements Component {
    protected Component component;
    public Decorator(Component component) {
        this.component = component;
    }
    public void operation() {
        component.operation();
    }
}
public class ConcreteDecorator extends Decorator {
    public ConcreteDecorator(Component component) {
        super(component);
    }
    public void operation() {
        super.operation();
        addedBehavior();
    }
    private void addedBehavior() {
        System.out.println("装饰器添加的行为");
    }
}
```

225. **代理模式的实现？** (参考链接)

- **静态代理**：

```java
public interface Subject {
    void request();
}
public class RealSubject implements Subject {
    public void request() {
        System.out.println("真实主题请求");
    }
}
public class Proxy implements Subject {
    private RealSubject realSubject;
    public Proxy() {
        this.realSubject = new RealSubject();
    }
    public void request() {
        System.out.println("代理前置处理");
        realSubject.request();
        System.out.println("代理后置处理");
    }
}
```

- **JDK动态代理**：

```java
public class DynamicProxy implements InvocationHandler {
    private Object target;
    public DynamicProxy(Object target) {
        this.target = target;
    }
    public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {
        System.out.println("动态代理前置处理");
        Object result = method.invoke(target, args);
        System.out.println("动态代理后置处理");
        return result;
    }
    public static Object getProxy(Object target) {
        return Proxy.newProxyInstance(
            target.getClass().getClassLoader(),
            target.getClass().getInterfaces(),
            new DynamicProxy(target)
        );
    }
}
```

- **CGLIB代理**：
  - 针对没有实现接口的类
  - 通过继承实现

226. **策略模式的使用？** (参考链接)

- **定义**：
  - 定义一系列算法，封装它们并使其可互换
  - 算法的变化不会影响使用算法的客户
- **应用场景**：
  - 多种支付方式：支付宝、微信、银联
  - 排序算法：快速排序、冒泡排序、归并排序
  - 不同促销策略：满减、折扣、积分
  - Comparator接口
- **实现**：

```
public interface Strategy {
    int doOperation(int num1, int num2);
}
public class OperationAdd implements Strategy {
    public int doOperation(int num1, int num2) {
        return num1 + num2;
    }
}
public class Context {
    private Strategy strategy;
    public Context(Strategy strategy) {
        this.strategy = strategy;
    }
    public int executeStrategy(int num1, int num2) {
        return strategy.doOperation(num1, num2);
    }
}
```

227. **模板方法模式的应用？** (参考链接)

- **定义**：
  - 定义算法骨架
  - 将一些步骤延迟到子类实现
- **应用场景**：
  - 数据库访问：

        - 获取连接 → 执行SQL → 处理结果 → 关闭连接
    - Spring JdbcTemplate
    - HttpServlet中service方法
    - 单元测试框架setUp/tearDown
- **实现**：

```
public abstract class AbstractClass {
    public final void templateMethod() {
        primitiveOperation1();
        primitiveOperation2();
        hook();
    }
    protected abstract void primitiveOperation1();
    protected abstract void primitiveOperation2();
    protected void hook() {} // 钩子方法
}
public class ConcreteClass extends AbstractClass {
    protected void primitiveOperation1() {
        System.out.println("具体类实现操作1");
    }
    protected void primitiveOperation2() {
        System.out.println("具体类实现操作2");
    }
}
```

228. **适配器模式的实现？** (参考链接)

- **类适配器**：

```java
public class Adaptee {
    public void specificRequest() {
        System.out.println("被适配者方法");
    }
}
public interface Target {
    void request();
}
public class ClassAdapter extends Adaptee implements Target {
    public void request() {
        specificRequest();
    }
}
```

- **对象适配器**：

```java
public class ObjectAdapter implements Target {
    private Adaptee adaptee;
    public ObjectAdapter(Adaptee adaptee) {
        this.adaptee = adaptee;
    }
    public void request() {
        adaptee.specificRequest();
    }
}
```

- **应用场景**：
  - Java I/O中的InputStreamReader
  - JDBC驱动程序
  - Spring AOP中的AdvisorAdapter

229. **建造者模式的使用场景？** (参考链接)

- **定义**：
  - 将复杂对象的构建与其表示分离
  - 同样的构建过程可以创建不同的表示
- **使用场景**：
  - 创建复杂对象：
    - SQL查询对象：SELECT * FROM table WHERE ... ORDER BY ...
    - HTTP请求对象

    - 参数较多的对象创建
    - 需要分步骤构建的对象
- **实现**：

```java
public class Computer {
    private String cpu;
    private String memory;
    private String disk;

    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.memory = builder.memory;
        this.disk = builder.disk;
    }

    public static class Builder {
        private String cpu;
        private String memory;
        private String disk;

        public Builder setCpu(String cpu) {
            this.cpu = cpu;
            return this;
        }
        public Builder setMemory(String memory) {
            this.memory = memory;
            return this;
        }
        public Builder setDisk(String disk) {
            this.disk = disk;
            return this;
        }
        public Computer build() {
            return new Computer(this);
        }
    }
}
```

230. **原型模式的作用？** (参考链接)

- **定义**：
    - 通过复制现有对象来创建新对象

    - 不需要知道创建细节
- **作用**：
    - 性能优化：避免重复初始化
    - 隐藏创建细节
    - 适用于创建成本大的对象
- **实现**：

```java
public class Prototype implements Cloneable {
    private String name;
    private List<String> list;

    public Prototype clone() {
        try {
            // 浅克隆
            Prototype cloned = (Prototype) super.clone();
            // 深克隆需要手动处理引用类型
            cloned.list = new ArrayList<>(this.list);
            return cloned;
        } catch (CloneNotSupportedException e) {
            throw new RuntimeException(e);
        }
    }
}
```

- **应用场景**：
    - 需要创建大量相似对象
    - 创建对象成本较大
    - 避免复杂的初始化过程

13. 项目经验考察（10题）

231. **请介绍一个你最有成就感的项目？**

- **项目背景**：
    - 项目名称、业务场景
    - 解决什么问题
    - 项目规模（用户量、数据量、QPS等）
- **我的角色**：
    - 承担的具体职责

    - 技术栈选择
    - 核心功能模块开发
- **技术难点**：
    - 遇到的技术挑战
    - 解决方案
    - 创新点
- **项目成果**：
    - 性能提升数据
    - 业务价值
    - 团队协作经验
- **个人成长**：
    - 技术能力提升
    - 项目管理经验
    - 沟通协作能力

232. **你在项目中遇到的最大技术挑战是什么？**

- **挑战描述**：
    - 具体问题场景
    - 技术难点分析
    - 对系统的影响
- **解决思路**：
    - 问题定位过程
    - 技术方案调研
    - 方案对比选择
- **实施方案**：
    - 具体实现步骤
    - 技术选型依据
    - 风险控制措施
- **解决效果**：
    - 性能提升数据
    - 问题解决程度
    - 对后续开发的影响
- **经验总结**：
    - 技术收获
    - 方法论沉淀
    - 避坑指南

233. **如何优化系统性能？**

- **性能分析**：
  - 使用性能分析工具（如JProfiler、Arthas）
  - 分析GC日志、慢查询日志
  - 识别性能瓶颈
- **数据库优化**：
  - SQL优化：索引优化、查询语句优化
  - 连接池优化：合理配置连接数
  - 分库分表：按业务维度拆分
  - 读写分离：缓解数据库压力
- **缓存优化**：
  - 合理使用缓存：减少数据库访问
  - 缓存策略：缓存更新策略、失效策略
  - 多级缓存：本地缓存+分布式缓存
- **应用层优化**：
  - 代码优化：减少循环嵌套、避免重复计算
  - 并发优化：合理使用线程池
  - 算法优化：选择更高效的算法
- **架构优化**：
  - 服务拆分：微服务化
  - 异步处理：消息队列削峰填谷
  - 负载均衡：分散请求压力

234. **如何处理高并发场景?**

- **系统架构**：
  - 水平扩展：增加服务器实例
  - 负载均衡：分发请求到多台服务器
  - 无状态设计：便于水平扩展
- **缓存策略**：
  - 多级缓存：CDN→Redis→本地缓存
  - 缓存预热：提前加载热点数据
  - 缓存降级：保护核心服务
- **限流降级**：
  - 限流：控制请求频率
  - 降级：关闭非核心功能
  - 熔断：隔离故障服务
- **异步处理**：
  - 消息队列：异步处理非核心业务

    - 削峰填谷：平滑请求波峰
- **数据库优化**：
    - 读写分离：分担数据库压力
    - 分库分表：提升并发能力
    - 连接池优化：合理配置参数

235. **如何保证系统的稳定性?**

- **容错设计**：
    - 熔断机制：防止故障扩散
    - 降级策略：保证核心功能可用
    - 限流保护：防止系统过载
- **监控告警**：
    - 应用监控：JVM、GC、线程池
    - 业务监控：核心指标、异常率
    - 基础设施监控：CPU、内存、网络
- **容灾备份**：
    - 数据备份：定期备份重要数据
    - 多机房部署：异地容灾
    - 故障演练：定期进行故障演练
- **代码质量**：
    - 代码审查：严格的Code Review
    - 单元测试：保证代码质量
    - 集成测试：验证系统功能
- **发布管理**：
    - 灰度发布：逐步放量
    - 回滚机制：快速回退
    - 发布验证：发布后验证

236. **如何进行系统监控?**

- **监控指标**：
    - 应用层：QPS、响应时间、错误率
    - JVM：堆内存使用率、GC频率、线程数
    - 数据库：连接数、慢查询、锁等待
    - 系统层：CPU、内存、磁盘IO、网络
- **监控工具**：
    - 应用监控：SkyWalking、CAT、Zipkin
    - 基础设施：Prometheus + Grafana

    - 日志监控：ELK Stack
- **告警机制**：
    - 阈值设定：合理设置告警阈值
    - 告警分级：严重级别分类
    - 告警渠道：短信、电话、IM
- **链路追踪**：
    - 全链路追踪：定位性能瓶颈
    - 业务链路：追踪业务流程
- **监控大盘**：
    - 实时监控：关键指标展示
    - 历史趋势：性能趋势分析

237. **如何进行系统容量规划?**

- **业务预估**：
    - 业务增长预期：用户量、交易量增长
    - 活动影响：大促活动流量预估
    - 季节性波动：考虑业务特点
- **性能测试**：
    - 基准测试：系统基础性能
    - 压力测试：极限承载能力
    - 稳定性测试：长时间运行性能
- **资源评估**：
    - 计算资源：CPU、内存需求
    - 存储资源：磁盘、数据库容量
    - 网络资源：带宽需求
- **扩容策略**：
    - 预留冗余：考虑业务增长
    - 弹性扩容：自动扩缩容
    - 成本控制：平衡性能与成本
- **监控预警**：
    - 容量预警：提前预警资源不足
    - 动态调整：根据实际使用调整

238. **如何进行故障排查?**

- **故障定位**：
    - 现象分析：收集故障现象
    - 影响范围：确定影响的用户/功能

    - 时间线：梳理故障发生时间线
- **日志分析**：
    - 应用日志：查找异常信息
    - 系统日志：检查系统错误
    - 数据库日志：查询慢SQL
- **监控数据**：
    - 性能指标：CPU、内存、网络变化
    - 业务指标：错误率、响应时间
    - 依赖服务：检查上下游服务状态
- **工具使用**：
    - 在线诊断：Arthas、jstack
    - 性能分析：JProfiler、JConsole
    - 网络分析：tcpdump、wireshark
- **根因分析**：
    - 逐层排查：从应用到基础设施
    - 假设验证：验证可能的原因
    - 修复验证：确认修复效果

239. **如何进行代码Review?**

- **Review标准**：
    - 代码规范：遵循团队编码规范
    - 设计模式：是否合理使用设计模式
    - 性能考虑：是否存在性能问题
    - 安全性：是否存在安全漏洞
- **Review重点**：
    - 业务逻辑：实现是否正确
    - 边界条件：是否考虑边界情况
    - 异常处理：异常处理是否完善
    - 可读性：代码是否易读易懂
- **Review流程**：
    - 提交前自检：开发者自查
    - 同事互审：同行评审
    - 专家审核：资深工程师把关
- **工具支持**：
    - 代码评审工具：Gerrit、GitLab MR
    - 静态分析：SonarQube、Checkstyle
    - 自动化检测：CI/CD集成

- **效果评估**：
  - 缺陷发现：及时发现潜在问题
  - 知识分享：促进团队学习
  - 质量提升：提高代码质量

240. **如何进行技术选型?**

- **需求分析**：
  - 业务需求：功能需求、性能需求
  - 非功能需求：可扩展性、可维护性
  - 约束条件：预算、时间、人员技能
- **技术评估**：
  - 功能对比：是否满足需求
  - 性能对比：吞吐量、响应时间
  - 社区活跃度：文档、社区支持
  - 生态成熟度：配套工具链
- **风险评估**：
  - 技术风险：学习成本、稳定性
  - 业务风险：是否影响业务发展
  - 团队风险：团队技能匹配度
- **试点验证**：
  - PoC验证：概念验证
  - 小规模试用：降低风险
  - 性能测试：验证性能指标
- **决策标准**：
  - 长期规划：考虑未来发展
  - 团队能力：匹配团队技术水平
  - 维护成本：考虑长期维护成本

14. 高并发系统设计（10题）

241. **如何设计一个高可用系统?**

- **架构层面**：
  - 多副本部署：主备、主从、集群模式
  - 负载均衡：分散请求压力
  - 服务拆分：微服务化，降低故障影响范围
  - 无状态设计：便于水平扩展和故障恢复

- **容错设计**：
  - 熔断机制：防止故障扩散
  - 限流降级：保护核心服务
  - 重试机制：处理临时故障
  - 超时控制：避免资源长时间占用
- **监控告警**：
  - 健康检查：实时监控服务状态
  - 指标监控：性能指标、业务指标
  - 告警机制：及时发现问题
  - 故障演练：定期验证高可用能力
- **容灾备份**：
  - 数据备份：定期备份重要数据
  - 多机房部署：异地多活
  - 快速恢复：自动化恢复机制

242. **如何进行系统限流?**

- **限流维度**：
  - 接口限流：单接口访问频率限制
  - 用户限流：单用户访问频率限制
  - IP限流：单IP访问频率限制
  - 应用限流：单应用访问频率限制
- **限流算法**：
  - 计数器算法：固定时间窗口内统计请求数
  - 滑动窗口算法：更精确的时间窗口控制
  - 漏桶算法：恒定速率处理请求
  - 令牌桶算法：支持突发流量处理
- **实现方式**：
  - 单机限流：基于内存计数器
  - 分布式限流：基于Redis等存储
  - 网关限流：统一接入层限流
  - 应用层限流：代码中实现限流

243. **如何进行系统降级?**

- **降级策略**：
  - 页面降级：返回静态页面或提示信息
  - 延迟服务：异步处理非紧急业务
  - 写降级：只读不写

    - 读降级：返回缓存数据
- **降级原则**：
    - 保证核心功能：优先保证核心业务可用
    - 优雅降级：提供友好的降级提示
    - 按优先级降级：先降级非核心功能
- **降级实现**：
    - 配置中心：动态控制降级开关
    - 代码实现：通过开关控制功能
    - 服务治理：框架层面支持降级
- **降级场景**：
    - 服务器压力过大
    - 某些服务异常
    - 第三方服务不稳定

244. **如何进行系统容灾?**

- **容灾等级**：
    - 数据容灾：保证数据不丢失
    - 应用容灾：保证服务可用
    - 地域容灾：跨地域容灾
- **容灾方案**：
    - 主备模式：备用系统平时不工作
    - 主主模式：双活系统同时提供服务
    - 多活模式：多地同时提供服务
- **数据同步**：
    - 同步复制：保证数据强一致性
    - 异步复制：保证性能优先
    - 半同步复制：平衡一致性与性能
- **故障切换**：
    - 自动切换：故障时自动切换
    - 手动切换：人工确认后切换
    - 切换验证：切换后验证服务状态
- **演练机制**：
    - 定期演练：验证容灾方案有效性
    - 专项演练：针对特定场景
    - 全链路演练：端到端验证

245. **如何进行系统扩容?**

- **扩容类型**：
  - 垂直扩容：增加单机资源（CPU、内存、磁盘）
  - 水平扩容：增加机器数量
  - 数据库扩容：分库分表
- **扩容策略**：
  - 预测扩容：根据业务增长预测扩容
  - 弹性扩容：根据负载自动扩容
  - 手动扩容：运维人员手动扩容
- **扩容考虑**：
  - 业务影响：是否需要停机
  - 数据迁移：数据同步策略
  - 服务发现：新实例注册
  - 负载均衡：流量分发策略
- **扩容流程**：
  - 容量评估：评估扩容需求
  - 资源申请：申请服务器资源
  - 环境部署：部署应用环境
  - 服务注册：注册到服务发现
  - 流量切换：逐步切换流量

246. **如何处理热点数据?**

- **热点识别**：
  - 监控分析：通过监控发现热点数据
  - 业务分析：从业务特点分析热点
  - 访问统计：统计访问频次
- **缓存策略**：
  - 多级缓存：本地缓存+Redis+CDN
  - 缓存预热：提前加载热点数据
  - 缓存更新：及时更新缓存数据
- **分流策略**：
  - 数据复制：多副本存储热点数据
  - 请求分片：将请求分散到不同节点
  - 读写分离：将读请求分散
- **降级策略**：
  - 缓存降级：热点数据降级处理
  - 限流保护：限制热点数据访问频率
  - 异步处理：异步更新热点数据

247. **如何处理缓存穿透?**

- **问题定义**:
  - 查询不存在的数据，缓存和数据库都没有
  - 每次查询都打到数据库
- **解决方案**:
  - **布隆过滤器**:
    - 在缓存前加一层过滤
    - 快速判断数据是否存在
    - 有一定的误判率
  - **缓存空值**:
    - 将空结果也缓存
    - 设置较短的过期时间
    - 防止恶意查询
  - **参数校验**:
    - 严格校验请求参数
    - 过滤非法请求
  - **实时监控**:
    - 监控异常查询
    - 识别攻击行为

248. **如何处理缓存雪崩?**

- **问题定义**:
  - 大量缓存key同时过期
  - 导致数据库压力过大
- **解决方案**:
  - **过期时间随机化**:
    - 在基础过期时间上加随机值
    - 避免同时过期
  - **缓存预热**:
    - 系统启动时提前加载热点数据
    - 避免缓存空窗期
  - **多级缓存**:
    - 本地缓存+Redis缓存
    - 提高缓存可用性
  - **服务降级**:
    - 缓存失效时返回默认值

        - 保证核心功能可用
    - **熔断机制**：
        - 数据库压力过大时熔断部分请求
        - 保护数据库

249. **如何进行系统监控?**

- **监控维度**：
    - 基础设施监控：CPU、内存、磁盘、网络
    - 应用监控：JVM、GC、线程池、连接池
    - 业务监控：核心指标、转化率、成功率
    - 用户体验监控：页面加载时间、错误率
- **监控工具**：
    - 基础监控：Prometheus、Zabbix
    - 应用监控：SkyWalking、CAT
    - 日志监控：ELK Stack
    - 链路追踪：Zipkin、Jaeger
- **告警机制**：
    - 阈值告警：达到阈值时告警
    - 趋势告警：异常趋势告警
    - 多级告警：不同级别不同通知方式
- **监控大盘**：
    - 实时监控：关键指标展示
    - 历史趋势：性能趋势分析
    - 对比分析：多维度对比

250. **如何进行系统压测?**

- **压测准备**：
    - 压测环境：与生产环境一致的测试环境
    - 压测数据：准备充足的测试数据
    - 压测脚本：模拟真实用户行为
- **压测工具**：
    - JMeter：功能强大，图形化界面
    - Gatling：基于Scala，性能好
    - wrk：轻量级，高并发测试
    - 自研工具：根据业务定制
- **压测指标**：
    - 吞吐量：QPS、TPS

    - 响应时间：平均响应时间、P95、P99
    - 错误率：请求错误比例
    - 资源使用率：CPU、内存、磁盘
- **压测策略**：
    - 基准测试：系统基础性能
    - 压力测试：极限承载能力
    - 稳定性测试：长时间运行性能
    - 容量测试：确定系统容量
- **压测分析**：
    - 性能瓶颈：识别系统瓶颈
    - 资源使用：分析资源使用情况
    - 优化建议：提出优化方案

15. AI相关技术（20题）

251. **什么是Transformer架构？其自注意力机制的原理是什么？** (参考链接)

- **Transformer架构**：
    - 2017年Google提出，完全基于注意力机制的神经网络架构
    - 由编码器（Encoder）和解码器（Decoder）组成
    - 编码器：N个相同的层堆叠，每层包含多头自注意力和前馈神经网络
    - 解码器：N个相同的层堆叠，每层包含掩码多头自注意力、编码器-解码器注意力和前馈神经网络
- **自注意力机制**：
    - 计算Query、Key、Value矩阵：Q=XW_Q, K=XW_K, V=XW_V
    - 注意力分数：Attention(Q,K,V) = softmax(QK^T/√d_k)V
    - 计算每个位置与其他位置的相关性得分
    - 通过加权求和得到输出表示
- **多头注意力**：
    - 并行计算多个注意力头
    - 捕捉不同子空间的信息
    - 拼接后通过线性变换输出

252. **如何在Java中集成AI模型？有哪些常用的Java AI框架？** (参考链接)

- **集成方式**：
    - 使用Java AI框架：直接在Java中训练和推理
    - 调用Python模型：通过REST API或进程调用

    - 模型转换：将模型转换为Java支持的格式
- **Java AI框架**：
    - **DL4J (DeepLearning4J)**：
        - Java原生深度学习库
        - 集成ND4J数值计算库
        - 支持多种神经网络架构
    - **TensorFlow Java API**：
        - Google官方Java绑定
        - 支持模型加载和推理
        - 与TensorFlow生态系统集成
    - **ONNX Runtime Java**：
        - 跨平台推理引擎
        - 支持多种模型格式
        - 高性能推理
    - **DJL (Deep Java Library)**：
        - Amazon开源的深度学习库
        - 支持MXNet、PyTorch、TensorFlow后端
        - 简化的API设计

253. **什么是LangChain4j？它在Java AI应用中的作用？** (参考链接)

- **定义**：
    - Java版LangChain，为Java开发者提供LLM开发框架
    - 简化大语言模型集成和应用开发
- **核心组件**：
    - **Model**：封装LLM服务（OpenAI、Anthropic、本地模型）
    - **Memory**：对话历史管理
    - **Output Parser**：结构化输出解析
    - **Tools**：外部工具集成
    - **Chains**：复合操作流程
- **应用场景**：
    - 聊天机器人开发
    - 文档问答系统
    - 文本生成应用
    - 智能对话系统

254. **什么是Spring AI？如何在Spring Boot中集成AI功能？** (参考链接)

- **Spring AI**：

    - Spring官方推出的AI开发框架
    - 简化AI模型集成和应用开发
    - 提供一致的API抽象
- **集成方式**：
    - 添加依赖：spring-ai-core、spring-ai-openai等
    - 配置模型参数：API Key、模型名称等
    - 使用Spring AI Template进行调用
- **主要功能**：
    - LLM抽象：统一的模型调用接口
    - Prompt工程：模板化提示词管理
    - 向量化：文档向量化和检索
    - 工具集成：外部工具调用

255. **什么是RAG（检索增强生成）系统？如何设计一个RAG系统？** (参考链接)

- **RAG定义**：
    - Retrieval-Augmented Generation，检索增强生成
    - 结合信息检索和文本生成的技术
    - 通过检索相关文档增强生成质量
- **系统组成**：
    - **索引阶段**：
        - 文档分割：将文档切分为块
        - 向量化：使用Embedding模型生成向量
        - 存储：保存到向量数据库
    - **检索阶段**：
        - 查询向量化：将用户查询转为向量
        - 相似度检索：在向量数据库中检索
        - 相关文档排序
    - **生成阶段**：
        - 构造Prompt：结合检索文档和查询
        - 调用LLM：生成最终答案
- **设计要点**：
    - 文档切分策略
    - 向量模型选择
    - 检索算法优化
    - Prompt设计

256. **向量数据库在AI应用中的作用？常用的向量数据库有哪些？** (参考链接)

- **作用**：
  - 存储高维向量数据
  - 支持相似度搜索
  - 实现语义检索
  - 提供快速近似最近邻搜索
- **常用数据库**：
  - **Milvus**：
    - 开源向量数据库
    - 支持多种索引类型
    - 高性能和可扩展性
  - **Pinecone**：
    - 托管向量数据库服务
    - 易于使用和管理
    - 自动索引优化
  - **Weaviate**：
    - 支持向量和结构化数据
    - GraphQL API
    - 内置机器学习模型
  - **Faiss**：
    - Facebook开源库
    - 高效相似度搜索
    - 支持多种索引算法
  - **Elasticsearch**：
    - 支持向量搜索
    - 全文检索能力
    - 丰富的生态系统

257. **如何处理大模型的上下文窗口限制问题？** (参考链接)

- **问题分析**：
  - 大模型上下文长度有限（如GPT-4最大128K tokens）
  - 限制了可处理的输入文本长度
  - 影响长文档处理能力
- **解决方案**：
  - **文档切分**：
    - 按语义单元切分文档
    - 保持上下文连贯性
    - 添加重叠片段

- **摘要提取**：
  - 预处理阶段生成摘要
  - 减少输入长度
  - 保留关键信息
- **检索增强**：
  - 使用RAG技术
  - 只处理相关片段
  - 减少上下文压力
- **滑动窗口**：
  - 分段处理长文本
  - 保持一定的上下文重叠
  - 合并处理结果

258. **什么是AI Agent？ReAct和Reflexion模型的区别？** (参考链接)

- **AI Agent**：
  - 具备感知、推理、规划和行动能力的智能体
  - 可以自主完成复杂任务
  - 通过与环境交互实现目标
- **ReAct模型**：
  - Reasoning + Acting
  - 在推理过程中生成文本和行动
  - 结合认知和行为
  - 适用于需要推理和工具调用的场景
- **Reflexion模型**：
  - 基于反思机制的强化学习
  - 通过自我反思改进行为
  - 从错误中学习
  - 更注重学习和适应能力

259. **如何在Java中使用TensorFlow或PyTorch模型进行推理？** (参考链接)

- **TensorFlow Java**：

```
// 加载模型
SavedModelBundle model = SavedModelBundle.load("/path/to/model", "serve");
Session session = model.session();

// 准备输入
Tensor<Float> input = Tensors.create(inputData);

// 执行推理
Tensor<?> result = session.runner()
    .feed("input_name", input)
    .fetch("output_name")
    .run()
    .get(0);
```

- **通过ONNX Runtime**：

```
OrtEnvironment env = OrtEnvironment.getEnvironment();
OrtSession.SessionOptions sessionOptions = new OrtSession.SessionOptions();
OrtSession session = env.createSession("model.onnx", sessionOptions);

// 创建输入张量
OnnxTensor inputTensor = OnnxTensor.createTensor(env, inputData);
OrtSession.Result result = session.run(Collections.singletonMap("input", inputTenso
```

- **使用DJL**：
  - 支持PyTorch、MXNet等后端
  - 统一的API接口
  - 自动内存管理

260. **DeepLearning4J (DL4J) 框架的使用和优化？** (参考链接)

- **基本使用**：
  - 构建神经网络：

```
MultiLayerConfiguration conf = new NeuralNetConfiguration.Builder()
    .updater(new Adam(0.01))
    .list()
    .layer(new DenseLayer.Builder().nIn(784).nOut(256).activation(Activation.REI
    .layer(new OutputLayer.Builder(LossFunctions.LossFunction.NEGATIVELOGLIKELIH
        .activation(Activation.SOFTMAX).nIn(256).nOut(10).build())
    .build();
MultiLayerNetwork model = new MultiLayerNetwork(conf);
model.init();
```

- **优化策略**：
  - **内存优化**：
    - 使用Workspace：减少内存分配
    - 合理设置batch size
  - **计算优化**：
    - 启用CuDNN（GPU）
    - 使用多线程数据加载
  - **模型优化**：
    - 模型量化
    - 网络剪枝

261. **如何在高并发场景下优化AI模型的推理性能？** (参考链接)

- **模型优化**：
  - **量化**：FP32→INT8，减少模型大小和推理时间
  - **剪枝**：去除冗余连接，减少计算量
  - **蒸馏**：训练小型学生模型
  - **图优化**：常量折叠、算子融合
- **推理优化**：
  - **批处理**：合并多个请求
  - **缓存**：缓存相似查询结果
  - **预加载**：模型预加载到内存
  - **异步处理**：非阻塞推理
- **系统优化**：
  - **GPU加速**：利用GPU并行计算
  - **负载均衡**：多实例部署
  - **资源隔离**：CPU绑核、内存限制

262. **AI模型服务的部署架构有哪些？如何与现有Java微服务集成？** (参考链接)

- **部署架构**：
  - **独立服务**：
    - 模型单独部署为REST API
    - 使用Flask/FastAPI或TorchServe
    - 通过HTTP调用
  - **嵌入式部署**：
    - 模型集成到Java应用
    - 使用ONNX Runtime或DJL
    - 直接API调用
  - **容器化部署**：
    - Docker容器部署
    - Kubernetes编排
    - 自动扩缩容
- **微服务集成**：
  - **同步调用**：
    - RestTemplate或Feign调用
    - 考虑超时和熔断
  - **异步处理**：
    - 消息队列处理推理请求
    - 适合非实时场景
  - **服务网格**：
    - Istio等服务网格
    - 流量管理和安全控制

263. **如何处理训练-推理不一致问题（Training-serving skew）？** (参考链接)

- **问题识别**：
  - 训练时性能良好，线上效果不佳
  - 数据分布发生变化
  - 特征处理不一致
- **解决方案**：
  - **数据一致性**：
    - 统一数据预处理逻辑
    - 特征工程线上线下一致
    - 数据版本管理
  - **特征存储**：
    - Feature Store统一管理
    - 训练推理使用相同特征

- **模型验证**：
  - 离线验证与在线A/B测试
  - 数据质量监控
- **监控告警**：
  - 性能指标监控
  - 数据漂移检测

264. **什么是Feature Store？在AI系统中的作用？** (参考链接)

- **定义**：
  - 统一管理机器学习特征的平台
  - 连接数据工程和模型训练/推理
- **核心功能**：
  - **特征存储**：
    - 存储计算好的特征
    - 版本控制和血缘追踪
  - **特征共享**：
    - 不同团队共享特征
    - 避免重复计算
  - **实时/离线统一**：
    - 同一套特征用于训练和推理
    - 保证一致性
- **作用**：
  - 提高特征复用性
  - 保证线上线下一致性
  - 加速模型开发迭代
  - 降低特征管理成本

265. **如何监控AI模型的性能和数据漂移？** (参考链接)

- **性能监控**：
  - **准确性指标**：
    - 精确率、召回率、F1分数
    - AUC、KS值等
    - 与基线对比
  - **系统指标**：
    - 推理延迟
    - 吞吐量
    - 资源使用率

- **数据漂移检测**：
  - **统计检验**：
    - KS检验、卡方检验
    - 监测分布变化
  - **特征漂移**：
    - 监测输入特征分布
    - 计算漂移指标
  - **概念漂移**：
    - 监测标签分布变化
    - 模型性能下降检测
- **监控工具**：
  - MLflow：实验和模型管理
  - Weights & Biases：训练监控
  - 自定义监控系统

266. **ONNX格式在模型部署中的优势？如何在Java中使用ONNX模型？** (参考链接)

- **ONNX优势**：
  - **互操作性**：
    - 跨框架兼容
    - 支持PyTorch、TensorFlow、Keras等
  - **优化**：
    - 图优化和算子融合
    - 自动优化推理性能
  - **标准化**：
    - 开放标准
    - 工业界广泛支持
- **Java使用**：

```
// 创建环境和会话
OrtEnvironment env = OrtEnvironment.getEnvironment();
OrtSession session = env.createSession("model.onnx");

// 准备输入
float[][] inputArray = {{1.0f, 2.0f, 3.0f}};
OnnxTensor inputTensor = OnnxTensor.createTensor(env, inputArray);

// 执行推理
OrtSession.Result result = session.run(Collections.singletonMap("input", inputTenso

// 获取输出
float[][] output = ((float[][])(result.get(0).getValue()));
```

267. **如何设计一个可扩展的AI模型服务API?** (参考链接)

- **API设计**:
  - **RESTful设计**:
    - 统一的请求/响应格式
    - 清晰的端点命名
    - 标准HTTP状态码
  - **请求格式**:
    - JSON格式输入
    - 支持批量推理
    - 参数验证
  - **响应格式**:
    - 结构化响应
    - 置信度分数
    - 处理时间
- **可扩展性**:
  - **无状态设计**:
    - 便于水平扩展
    - 负载均衡友好
  - **异步处理**:
    - 长耗时任务异步处理
    - 消息队列支持
  - **缓存策略**:
    - 相似请求结果缓存
    - 模型预测结果缓存

268. **什么是模型量化？在Java AI应用中的实践？** (参考链接)

- **模型量化**：
  - 将高精度模型（FP32）转换为低精度模型（INT8、INT4）
  - 减少模型大小和推理时间
  - 降低内存占用和能耗
- **量化类型**：
  - **训练后量化**：
    - 对已训练模型进行量化
    - 不需要重新训练
    - 精度损失可控
  - **量化感知训练**：
    - 在训练过程中模拟量化
    - 减少精度损失
    - 需要重新训练
- **Java实践**：
  - **ONNX Runtime**：支持INT8量化模型
  - **TensorFlow Lite**：通过转换器量化
  - **DJL**：支持量化模型推理
  - **性能提升**：通常2-4倍推理速度提升

269. **如何在Java应用中实现模型的A/B测试？** (参考链接)

- **A/B测试架构**：
  - **流量分配**：
    - 按用户ID、会话ID或随机分配
    - 确保实验组对照组可比
    - 流量分配策略
  - **模型路由**：
    - 通过模型版本控制
    - 动态路由策略
    - 灰度发布支持
- **实现方式**：
  - **服务层控制**：

```java
@Service
public class ModelService {
    public Result predict(Request request) {
        String modelVersion = getTrafficAllocation(request.getUserId());
        switch(modelVersion) {
            case "v1": return modelV1.predict(request);
            case "v2": return modelV2.predict(request);
            default: return modelV1.predict(request);
        }
    }
}
```

  - **中间件控制**：
    - 网关层路由
    - 服务网格控制
- **效果评估**：
  - **指标监控**：
    - 业务指标对比
    - 模型性能对比
    - 统计显著性检验

270. **AI模型的安全性考虑有哪些？如何防止模型投毒和对抗样本攻击？** (参考链接)

- **模型投毒**：
  - **定义**：
    - 在训练数据中注入恶意样本
    - 影响模型学习和预测
  - **防护措施**：
    - 数据清洗和验证
    - 异常检测算法
    - 鲁棒训练方法
- **对抗样本**：
  - **定义**：
    - 对输入添加微小扰动
    - 误导模型做出错误预测
  - **防护方法**：
    - 对抗训练
    - 输入预处理
    - 检测算法
- **安全实践**：

- **访问控制**：
  - 模型API访问控制
  - 身份认证和授权
- **输入验证**：
  - 输入数据合法性检查
  - 异常值检测
- **模型保护**：
  - 模型水印
  - 知识蒸馏保护
  - 模型加密

题目统计

- Java基础：30题（含答案）
- JVM：20题（含答案）
- 多线程与并发：30题（含答案）
- Java集合：20题（含答案）
- 数据库：25题（含答案）
- Redis：20题（含答案）
- 框架相关：25题（含答案）
- 分布式系统：20题（含答案）
- 系统设计：10题（含答案）
- 算法与数据结构：10题（含答案）
- 网络与操作系统：10题（含答案）
- 设计模式：10题（含答案）
- 项目经验考察：10题（含答案）
- 高并发系统设计：10题（含答案）
- AI相关技术：20题（含答案）

总计：270题（全部含答案）